

École Hassania des Travaux Publics

Electrical and Energy Engineering

Design and FPGA Implementation of a Real-Time Cable Harness Test Bench

for Industrial Fault Detection

Technical Report

Author:	Noureddine Bedoui
Technology:	FPGA / VHDL / MATLAB-Simulink / Vivado
Target Device:	AMD/Xilinx Artix-7 XC7A100T
Processing Clock:	40 MHz

Abstract

This project presents the design and FPGA implementation of a real-time digital signal-processing system for cable harness fault detection. MATLAB/Simulink is first used as a reference environment to model the signal-processing chain and evaluate the proposed detection strategy. The algorithm is then converted into VHDL using HDL Coder and implemented on an AMD/Xilinx Artix-7 XC7A100T FPGA.

The implemented processing chain includes FIR filtering, signal difference computation, absolute-value processing, gain, threshold comparison, and event validation. The FPGA design operates with a 40 MHz processing clock and uses signed 16-bit fixed-point arithmetic.

RTL simulation is performed using Vivado to verify the externally observable input-to-detection behavior. FPGA implementation results include timing analysis, resource utilization, power estimation, and design-rule checking.

Contents

1	Introduction	6
1.1	Industrial Context	6
1.2	Cable Harness Testing Problem	6
1.3	Motivation for FPGA-Based Real-Time Processing	7
1.4	Project Objectives	7
1.5	Proposed System Architecture	8
1.6	Development and Verification Methodology	8
1.7	Main Contributions	9
1.8	Report Organization	9
2	Overall System Architecture	10
2.1	System Overview	10
2.2	Cable Harness and Wire Selection	11
2.3	Switching and Multiplexing Architecture	12
2.4	Signal Conditioning and Analog-to-Digital Conversion	12
2.5	FPGA-Based Real-Time Processing	13
2.5.1	FIR Filtering	13
2.5.2	Difference Computation	14
2.5.3	Absolute-Value Operation	14
2.5.4	Gain and Threshold	14
2.5.5	Event Validation	15
2.6	FPGA Target and Clock Architecture	15
2.7	External FPGA Interface	15
2.8	Overall Processing Workflow	16
2.9	System-Level Summary	17
3	MATLAB/Simulink Reference Model and Parameter Selection	18
3.1	Reference Modeling Approach	18
3.2	Input Signal Generation	18
3.3	Fault Scenarios	19
3.4	Fixed-Point Representation	19
3.5	Digital Filtering	20
3.6	Difference Computation	20
3.7	Absolute-Value Processing	20
3.8	Gain Selection	20
3.8.1	Case 1: Gain = 10	21
3.8.2	Case 2: Gain = 25	21
3.8.3	Case 3: Gain = 50	22

3.8.4	Case 4: Gain = 100	23
3.8.5	Case 5: Gain = 1000	24
3.8.6	Gain Comparison	25
3.9	Threshold Selection	26
3.9.1	Case 1: Threshold = 5	26
3.9.2	Case 2: Threshold = 7.5	27
3.9.3	Case 3: Threshold = 9.5	27
3.9.4	Case 4: Threshold = 12	28
3.9.5	Case 5: Threshold = 15	29
3.9.6	Threshold Comparison	30
3.10	Final Parameter Selection	30
3.11	Reference Model Summary	31
4	FPGA Architecture and HDL Implementation	32
4.1	Target FPGA Device	32
4.2	MATLAB/Simulink to VHDL Generation Flow	33
4.3	Clock Architecture	33
4.4	Fixed-Point Hardware Representation	34
4.5	Real-Time Signal-Processing Chain	34
4.5.1	FIR Filtering	35
4.5.2	Difference Computation	35
4.5.3	Absolute-Value Operation	35
4.5.4	Gain Application	36
4.5.5	Threshold Comparison	36
4.5.6	Event Validation	36
4.6	FPGA External Interface	36
4.7	HDL and Vivado Integration	37
4.8	Implementation Configuration Summary	38
4.9	Chapter Summary	39
5	Functional Verification	40
5.1	Verification Objectives	40
5.2	MATLAB/Simulink Reference Model	41
5.3	Vivado RTL Simulation	42
5.4	Comparison Between MATLAB/Simulink and Vivado	43
5.5	Waveform Analysis	43
5.6	Verification Under Normal Operating Conditions	44
5.7	Verification of Fault-Related Events	44
5.8	Verification of Repeated Detection Events	44
5.9	Reset and Initialization Verification	45
5.10	Verification Coverage and Limitations	45
5.11	Chapter Summary	46
6	FPGA Implementation Results and Timing Closure	47
6.1	Initial Vivado Implementation	47
6.2	Initial Timing Failure	47
6.3	Diagnosis of the Timing Problem	48
6.4	Addition of the Clock Management Block	48
6.5	Transition to 40 MHz Processing	49

6.6	Re-Synthesis and Re-Implementation	49
6.7	Final Timing Closure	49
6.8	Resource Utilization	50
6.9	Power Analysis	51
6.10	Design Rule Check	51
6.11	TIMESTAMP Implementation Decision	52
6.12	Before and After Clock Modification	52
6.13	Discussion of Implementation Results	53
6.14	Chapter Summary	53
7	Discussion	55
7.1	Overall Discussion	55
7.2	Main Strengths and Limitations	55
7.3	Discussion Summary	56
8	Future Work	57
8.1	Hardware and Acquisition Improvements	57
8.2	FPGA Architecture Improvements	57
8.3	Detection Algorithm Improvements	57
8.4	System-Level Extensions	57
8.5	Future Work Summary	58
9	Conclusion	59

List of Figures

2.1	Overall architecture of the proposed cable harness test bench.	11
2.2	Implemented FPGA real-time signal-processing architecture.	13
3.1	MATLAB/Simulink simulation for $G = 10$	21
3.2	MATLAB/Simulink simulation for $G = 25$	22
3.3	MATLAB/Simulink simulation for $G = 50$	23
3.4	MATLAB/Simulink simulation for $G = 100$	24
3.5	MATLAB/Simulink simulation for $G = 1000$	25
3.6	MATLAB/Simulink simulation for $T = 5$ with $G = 50$	26
3.7	MATLAB/Simulink simulation for $T = 7.5$ with $G = 50$	27
3.8	MATLAB/Simulink simulation for $T = 9.5$ with $G = 50$	28
3.9	MATLAB/Simulink simulation for $T = 12$ with $G = 50$	29
3.10	MATLAB/Simulink simulation for $T = 15$ with $G = 50$	29
4.1	Model-based FPGA development flow from MATLAB/Simulink to VHDL generation and Vivado implementation.	33
4.2	FPGA clock architecture showing the 100 MHz input clock and generated 40 MHz processing clock.	33
4.3	Vivado FPGA implementation environment for the generated VHDL design.	38
5.1	MATLAB/Simulink reference simulation showing the input signal, pro- cessed signal, detection output, and event timing information.	41
5.2	Vivado RTL simulation waveform showing the FPGA input, control signals, and the externally observable detection output <code>Out3</code>	42
6.1	Initial Vivado implementation result before timing optimization.	47
6.2	Vivado post-implementation resource utilization report.	50
6.3	Vivado power analysis for the final FPGA implementation.	51
6.4	Comparison of Vivado timing results before and after the processing-clock modification.	53

List of Tables

2.1	Main externally observable FPGA signals.	16
3.1	Qualitative comparison of the tested gain values.	25
3.2	Qualitative comparison of the tested threshold values.	30
4.1	Main FPGA implementation parameters.	32
4.2	Main external FPGA interface signals.	37
4.3	Final FPGA processing configuration.	39
5.1	Main signals considered during RTL functional verification.	42
5.2	Functional correspondence between the reference model and FPGA implementation.	43
6.1	Final FPGA resource utilization.	50
6.2	Timing results before and after the clock modification.	52

Chapter 1

Introduction

1.1 Industrial Context

Cable harnesses are widely used in automotive, aerospace, industrial automation, and embedded systems to interconnect sensors, actuators, electronic control units, and power or communication subsystems. Their correct operation is essential because a defect affecting a single conductor can lead to intermittent communication, incorrect sensor measurements, malfunction of an electronic subsystem, or complete system failure.

As the number of conductors and interconnections increases, cable harness testing becomes more challenging. Conventional testing approaches generally rely on electrical continuity measurements and predefined inspection procedures. However, detecting transient or subtle electrical anomalies requires not only an appropriate measurement interface but also a reliable signal-processing and decision mechanism.

For industrial applications, the test system should ideally provide deterministic processing, low detection latency, and sufficient computational resources to process the acquired signal in real time. These requirements motivate the use of programmable hardware, particularly Field-Programmable Gate Arrays (FPGAs), for implementing dedicated signal-processing pipelines.

1.2 Cable Harness Testing Problem

A cable harness can be considered as a collection of individual conductors that must satisfy predefined electrical characteristics during operation or testing. A fault may appear as an abrupt variation in the measured electrical signal, a transient disturbance, or another abnormal waveform behavior.

A simplified conceptual measurement architecture consists of a signal or excitation source connected to the cable harness, followed by a switching or multiplexing stage used to select the conductor under test. The selected signal can then be conditioned and converted into a digital representation using an Analog-to-Digital Converter (ADC). The resulting digital samples are provided to a real-time processing unit responsible for detecting abnormal behavior.

In the proposed system, the switching and analog acquisition stages are considered part of the overall system architecture, while the main implemented contribution focuses on the digital signal-processing and FPGA stages.

The objective is therefore not simply to acquire a signal, but to transform the acquired samples into a reliable fault-detection decision while maintaining deterministic real-time

behavior.

1.3 Motivation for FPGA-Based Real-Time Processing

Digital signal processing can be implemented using general-purpose processors, micro-controllers, digital signal processors, or programmable logic. While software-based implementations provide flexibility, an FPGA can provide a dedicated hardware datapath in which several processing operations are executed with predictable timing.

For the proposed cable harness test bench, the FPGA is used to implement a fixed-point signal-processing chain consisting of digital filtering, signal-difference computation, absolute-value processing, gain, threshold comparison, and event validation.

The use of fixed-point arithmetic is particularly relevant for FPGA implementation because it allows the numerical representation to be explicitly defined and can reduce hardware resource requirements compared with floating-point implementations.

Another important motivation is the deterministic nature of FPGA processing. Once synthesized and implemented, the processing architecture operates according to a defined clock and hardware datapath, making it suitable for applications where predictable response time is important.

1.4 Project Objectives

The main objective of this project is to design and verify a real-time FPGA-based digital processing system for industrial cable harness fault detection.

The specific objectives are:

- Develop a reference signal-processing model using MATLAB/Simulink.
- Define a fixed-point numerical representation suitable for FPGA implementation.
- Develop a digital processing chain including FIR filtering, signal-difference computation, absolute-value processing, gain, threshold comparison, and event validation.
- Generate VHDL hardware description code from the MATLAB/Simulink model using HDL Coder.
- Implement the generated VHDL design on an AMD/Xilinx Artix-7 XC7A100T FPGA.
- Use AMD/Xilinx Vivado for RTL simulation, synthesis, implementation, timing analysis, resource utilization analysis, power estimation, and design-rule checking.
- Verify the externally observable input-to-detection behavior of the FPGA design through RTL simulation.
- Evaluate the timing, resource utilization, power estimation, and implementation characteristics of the resulting FPGA design.

1.5 Proposed System Architecture

The complete system is organized into several functional stages. At the conceptual system level, the cable harness is connected to a measurement interface that selects the conductor under test and provides the corresponding electrical signal to an acquisition stage.

The digital processing path begins after signal acquisition and conversion. The resulting digital samples are processed by the FPGA using the following sequence:

$$\text{Input} \rightarrow \text{FIR} \rightarrow \text{Difference} \rightarrow |\text{Difference}| \rightarrow \text{Gain} \rightarrow \text{Threshold} \rightarrow \text{Event Validation} \rightarrow \text{Detect} \quad (1.1)$$

The implemented FPGA design operates with a processing clock of 40MHz, corresponding to a clock period of 25ns. The main input signal is represented using signed 16-bit fixed-point arithmetic with the MATLAB/Simulink format

$$\text{fixdt}(1,16,12) \quad (1.2)$$

which corresponds to a scaling factor of

$$2^{12} = 4096. \quad (1.3)$$

The detection criterion implemented in the processing chain is based on the amplified absolute difference:

$$|\text{diff}| \times 50 > 9.5. \quad (1.4)$$

Equivalently, the threshold condition can be expressed as

$$|\text{diff}| > 0.19. \quad (1.5)$$

The final externally observable output of the FPGA design is the **Detect** signal.

1.6 Development and Verification Methodology

The project follows a model-based hardware development methodology. MATLAB/Simulink is first used to develop and analyze the signal-processing algorithm. This reference model provides the expected behavior of the digital processing chain before hardware implementation.

The resulting model is then converted into VHDL using HDL Coder. The generated HDL is subsequently integrated and analyzed using AMD/Xilinx Vivado.

The overall development flow is:

$$\text{MATLAB/Simulink} \rightarrow \text{HDL Coder} \rightarrow \text{VHDL} \rightarrow \text{Vivado RTL Simulation} \rightarrow \text{Synthesis} \rightarrow \text{Implementation} \quad (1.6)$$

This methodology allows the design to be evaluated at several levels. MATLAB/Simulink is used for algorithmic and numerical verification, while Vivado is used to verify the generated RTL behavior and evaluate the physical implementation characteristics of the FPGA design.

The verification process therefore considers both functional behavior and implementation-level characteristics.

1.7 Main Contributions

The main contributions of this project can be summarized as follows:

1. Development of a MATLAB/Simulink reference model for real-time cable harness fault detection.
2. Definition of a fixed-point digital signal-processing architecture suitable for FPGA implementation.
3. Generation of a VHDL implementation from the Simulink model using HDL Coder.
4. Implementation and analysis of the design on an AMD/Xilinx Artix-7 XC7A100T FPGA using Vivado.
5. RTL functional verification of the externally observable input-to-Detect behavior.
6. Evaluation of FPGA timing, resource utilization, power estimation, clocking, and design-rule-checking results.

The resulting implementation demonstrates the feasibility of realizing a compact real-time digital fault-detection datapath on an Artix-7 FPGA. The implementation results are analyzed in the following chapters to assess both the functional behavior and the hardware characteristics of the proposed architecture.

1.8 Report Organization

The remainder of this report is organized as follows.

Chapter 2 presents the overall system architecture and describes the conceptual cable harness measurement structure, conductor selection, signal acquisition, and FPGA processing stages.

Chapter 3 describes the MATLAB/Simulink reference model, including the input signal representation, fixed-point arithmetic, filtering, difference computation, thresholding, and event validation.

Chapter 4 presents the FPGA architecture and hardware implementation methodology, including the HDL Coder workflow, VHDL generation, clock architecture, and Vivado implementation flow.

Chapter 5 focuses on functional verification. The MATLAB/Simulink reference behavior and the Vivado RTL simulation results are analyzed, with particular attention to the externally observable input and Detect signals.

Chapter 6 presents the FPGA implementation results, including timing analysis, resource utilization, power estimation, clocking, and design-rule checking.

Chapter 7 discusses the obtained results and the main engineering implications of the implementation.

Chapter 8 presents possible improvements and future extensions of the system.

Finally, Chapter 9 summarizes the work and presents the main conclusions of the project.

Chapter 2

Overall System Architecture

2.1 System Overview

The proposed system is designed as a real-time cable harness testing and fault-detection platform. Its main purpose is to acquire an electrical signal associated with a selected cable conductor, process the resulting digital samples in real time, and generate a detection decision when an abnormal signal variation satisfies the predefined detection criteria.

The overall architecture can be divided into two main domains:

1. the conceptual measurement and acquisition domain, which includes conductor selection, signal conditioning, and analog-to-digital conversion;
2. the implemented digital processing domain, which is based on an FPGA and performs the real-time signal-processing and fault-detection operations.

A simplified representation of the complete system is shown in Figure 2.1.

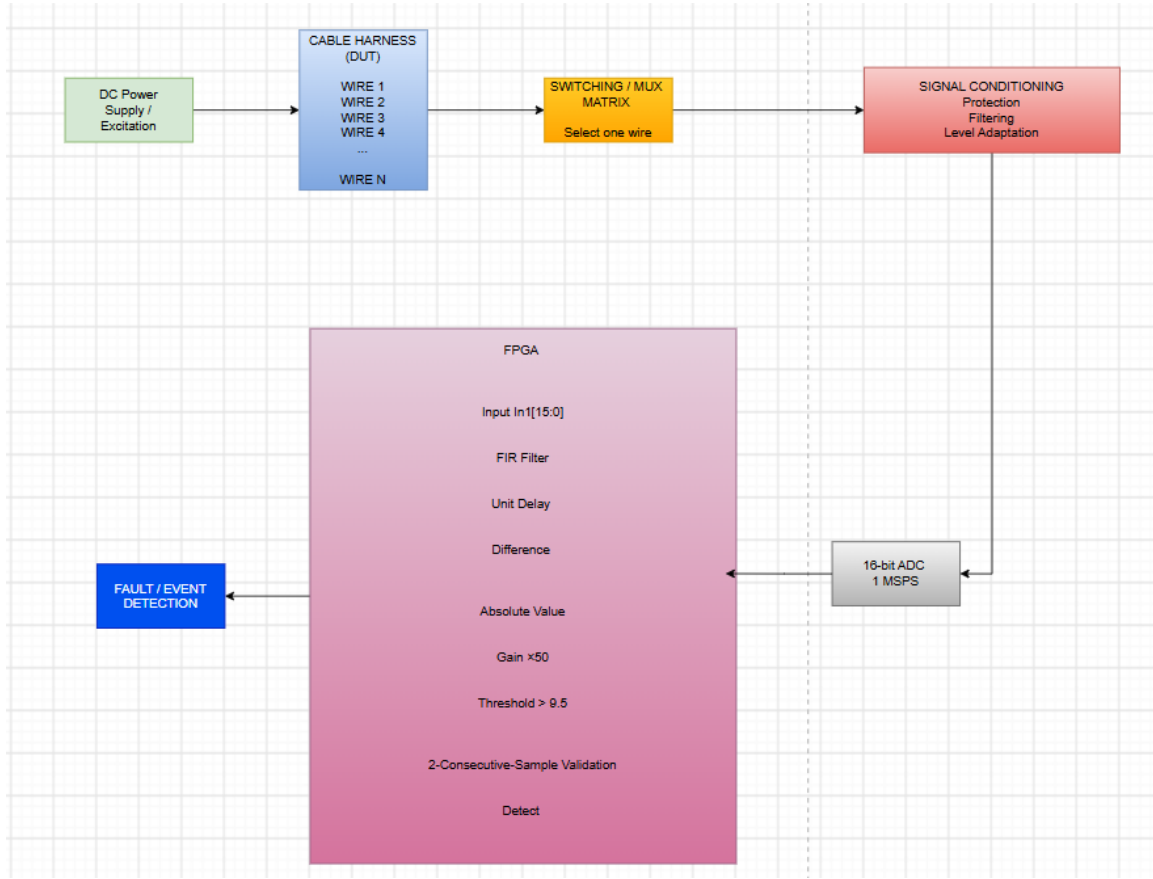


Figure 2.1: Overall architecture of the proposed cable harness test bench.

The conceptual signal path is:

$$\text{Excitation Source} \rightarrow \text{Cable Harness} \rightarrow \text{Switching/MUX} \rightarrow \text{Signal Conditioning} \rightarrow \text{ADC} \rightarrow \text{FPGA} \rightarrow \text{Fault / Event Detection} \quad (2.1)$$

The switching, signal-conditioning, and ADC stages define the interface between the physical cable harness and the digital processing system. The FPGA then operates on the digitized signal using the processing architecture described in the following sections.

It should be noted that the analog front-end and switching stages shown in this architecture represent the proposed system-level realization. The implemented contribution of this project focuses on the digital signal-processing and FPGA stages.

2.2 Cable Harness and Wire Selection

A cable harness consists of multiple conductors that may need to be tested individually. Testing all conductors simultaneously would increase the complexity and hardware requirements of the acquisition system. A practical alternative is to sequentially select one conductor at a time and connect it to a shared measurement chain.

Let the cable harness contain N conductors:

$$\mathcal{W} = \{W_1, W_2, \dots, W_N\}. \quad (2.2)$$

A switching or multiplexing stage can select one conductor W_i at a given time and route its signal toward the measurement interface.

The selection operation can therefore be represented as:

$$W_i \rightarrow \text{Measurement Chain}, \quad i \in \{1, \dots, N\}. \quad (2.3)$$

This architecture allows several conductors to share the same signal-conditioning and acquisition resources. Consequently, the number of analog acquisition channels does not necessarily need to match the total number of conductors in the harness.

The selection stage itself does not perform fault detection. Its function is to establish which conductor is currently connected to the measurement path.

2.3 Switching and Multiplexing Architecture

The switching stage provides the interface between the individual conductors and the common acquisition chain. A multiplexer (MUX) can be used to select one of several input signals according to a control signal.

For a system with N conductors, the MUX can be represented conceptually as

$$Y = W_k, \quad (2.4)$$

where k is the currently selected conductor.

The switching stage is particularly useful for cable harness testing because the same downstream signal-processing architecture can be reused for multiple conductors. This reduces the required hardware duplication and provides a scalable architecture.

In a practical implementation, the switching mechanism would be controlled by a test-management layer responsible for selecting the conductor under test and associating the resulting detection event with the corresponding wire identifier.

The present FPGA implementation does not include the physical switching hardware. Instead, the digital processing subsystem is designed to receive the digitized signal corresponding to the selected conductor.

2.4 Signal Conditioning and Analog-to-Digital Conversion

The electrical signal obtained from the selected conductor may require signal conditioning before digital conversion. Depending on the characteristics of the physical measurement, the conditioning stage can include attenuation, amplification, filtering, level shifting, or protection circuitry.

The conditioned signal is then converted into a digital representation by an Analog-to-Digital Converter (ADC).

For the proposed architecture, the digital processing model uses a sampling configuration associated with a 1MHz signal-processing dataset. The FPGA processing itself uses a signed 16-bit fixed-point representation.

The digital input is represented using:

$$\text{fixdt}(1, 16, 12). \quad (2.5)$$

This representation contains one sign bit, 15 remaining magnitude-related bits, and 12 fractional bits. The corresponding scaling factor is

$$S = 2^{12} = 4096. \quad (2.6)$$

Consequently, the digital processing chain operates on a deterministic fixed-point representation rather than floating-point values.

The ADC and analog front-end are considered at the system-architecture level in this project. The FPGA implementation starts from the digital input signal.

2.5 FPGA-Based Real-Time Processing

The FPGA constitutes the main digital processing unit of the proposed test bench. Its role is to process the acquired samples using a dedicated hardware datapath and generate a fault-detection signal.

The implemented FPGA processing chain is shown conceptually in Figure 2.2.

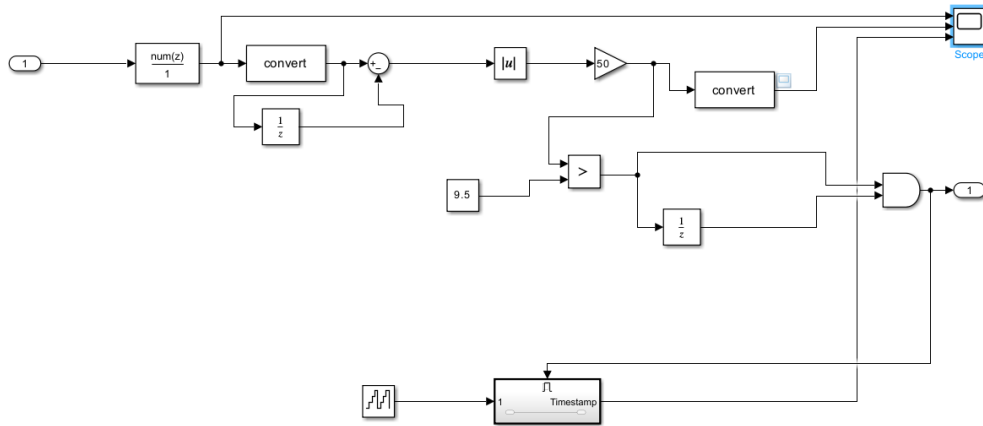


Figure 2.2: Implemented FPGA real-time signal-processing architecture.

The processing chain is composed of the following stages:

$$\text{In1} \rightarrow \text{FIR} \rightarrow \text{Difference} \rightarrow \text{Absolute Value} \rightarrow \text{Gain} \rightarrow \text{Threshold} \rightarrow \text{Event Validation} \rightarrow \text{Detect}. \quad (2.7)$$

Each stage performs a specific operation on the incoming signal.

2.5.1 FIR Filtering

The first stage of the digital processing chain is a Finite Impulse Response (FIR) filter.

The FIR stage is used to process the sampled input signal before the subsequent detection operations. Its purpose is to shape the signal and reduce unwanted high-frequency variations according to the reference model.

For an input sequence $x[n]$, the FIR output can generally be expressed as

$$y[n] = \sum_{k=0}^{M-1} h[k]x[n-k], \quad (2.8)$$

where $h[k]$ represents the filter coefficients and M is the filter length.

The FIR stage is implemented as part of the HDL-generated processing architecture.

2.5.2 Difference Computation

After filtering, the system evaluates the variation of the processed signal. The difference operation provides a measure of the change between successive processed values.

A simplified representation is

$$d[n] = y[n] - y[n - 1]. \quad (2.9)$$

This operation allows abrupt signal variations to be emphasized while relatively stable signal regions produce small difference values.

2.5.3 Absolute-Value Operation

Because the detection mechanism is concerned with the magnitude of the signal variation rather than its direction, the absolute value of the difference is calculated:

$$a[n] = |d[n]|. \quad (2.10)$$

Both positive and negative signal transitions can therefore contribute to the detection criterion.

2.5.4 Gain and Threshold

The absolute difference is multiplied by a gain factor of 50:

$$g[n] = 50|d[n]|. \quad (2.11)$$

The resulting value is then compared with a threshold of 9.5.

The detection condition is therefore

$$g[n] > 9.5. \quad (2.12)$$

Equivalently,

$$50|d[n]| > 9.5, \quad (2.13)$$

which gives

$$|d[n]| > 0.19. \quad (2.14)$$

The comparator consequently produces a qualifying event when the processed signal variation exceeds the defined threshold.

2.5.5 Event Validation

A validation stage is included after the threshold comparison to prevent an isolated qualifying sample from being immediately interpreted as a valid event.

The implemented detection strategy uses consecutive qualifying samples as part of the event-validation mechanism. In conceptual form, if $q[n]$ represents the threshold-comparator output, the validation logic evaluates consecutive values of $q[n]$ before asserting the final detection output.

The externally observable output of this stage is the **Detect** signal.

The available RTL verification demonstrates the response of the external **Detect** output to the applied input waveforms. However, the internal validation state is not externally exposed in the current top-level interface. Therefore, detailed internal verification of the validation state is discussed as a limitation in Chapter 5.

2.6 FPGA Target and Clock Architecture

The digital processing system is implemented for an AMD/Xilinx Artix-7 XC7A100T FPGA.

The selected device is:

- Device family: AMD/Xilinx Artix-7
- Device: XC7A100T
- Package: CSG324
- Speed grade: -1

The FPGA design receives a 100MHz system clock. A clock-management block generates a 40MHz processing clock used by the main processing logic.

The processing clock period is therefore

$$T_{\text{proc}} = \frac{1}{40 \times 10^6} = 25 \text{ ns.} \quad (2.15)$$

The clock relationship can be summarized as

$$100\text{MHz} \rightarrow \text{Clock Management} \rightarrow 40\text{MHz}. \quad (2.16)$$

The 40MHz clock provides the timing reference for the implemented processing datapath.

The detailed timing results obtained after Vivado implementation are presented in Chapter 6.

2.7 External FPGA Interface

The externally observable interface of the implemented FPGA processing design contains the following main signals:

Table 2.1: Main externally observable FPGA signals.

Signal	Direction	Description
In1[15:0]	Input	Signed 16-bit input signal
clk	Input	System clock
reset	Input	Reset control
clk_enable	Input	Processing clock-enable control
ce_out	Output	Clock-enable output
Out3	Output	Fault-detection output

The `Out3` signal corresponds to the externally observable detection output of the implemented design and is referred to as `Detect` throughout this report.

Internal processing signals such as the FIR output, difference signal, absolute-value output, gain output, comparator output, and event-validation state are not exposed at the top-level interface.

This distinction is important for the verification methodology. The available RTL waveforms can directly establish the relationship between the applied input and the final detection output, but they do not independently expose every intermediate processing stage.

2.8 Overall Processing Workflow

The complete development and implementation workflow combines system-level modeling with hardware implementation.

The first stage consists of developing the signal-processing algorithm in MATLAB/Simulink. The model provides a reference representation of the intended behavior and allows the fixed-point processing chain to be evaluated before hardware generation.

The model is then converted into VHDL using HDL Coder. The generated HDL is subsequently simulated and implemented using AMD/Xilinx Vivado.

The overall workflow is summarized as follows:

1. Define the cable harness testing concept and measurement architecture.
2. Generate and analyze representative digital input signals.
3. Develop the signal-processing algorithm in MATLAB/Simulink.
4. Configure the model using the selected fixed-point representation.
5. Generate VHDL using HDL Coder.
6. Integrate the generated design into the Vivado environment.
7. Perform RTL simulation and analyze the externally observable input-to-`Detect` behavior.
8. Perform synthesis and implementation on the target Artix-7 FPGA.
9. Analyze timing, resource utilization, power estimation, clocking, and DRC results.

This workflow establishes a traceable relationship between the reference model, generated HDL, RTL simulation, and FPGA implementation.

2.9 System-Level Summary

The proposed architecture combines a conceptual cable harness measurement interface with an implemented FPGA-based digital processing subsystem.

At the system level, the conductor under test is selected and its electrical signal is assumed to be conditioned and converted into digital samples. At the digital level, the FPGA processes the samples using a fixed-point FIR-based detection architecture.

The principal implemented signal path is:

$$\boxed{\text{In1}[15:0] \rightarrow \text{FIR} \rightarrow \Delta \rightarrow |\Delta| \rightarrow \times 50 \rightarrow > 9.5 \rightarrow \text{Validation} \rightarrow \text{Detect}} \quad (2.17)$$

The use of MATLAB/Simulink as the reference-modeling environment, HDL Coder for VHDL generation, and Vivado for RTL simulation and FPGA implementation provides a complete model-to-hardware development flow.

The next chapter focuses on the MATLAB/Simulink reference model and explains how the numerical representation and signal-processing operations are defined before hardware implementation.

Chapter 3

MATLAB/Simulink Reference Model and Parameter Selection

3.1 Reference Modeling Approach

Before the FPGA implementation, a behavioral reference model was developed in MATLAB/Simulink to define and evaluate the proposed digital signal-processing and fault-detection algorithm.

The reference model serves two main purposes. First, it provides a convenient environment for developing and analyzing the detection algorithm under controlled simulation conditions. Second, it establishes a functional reference for the subsequent HDL generation and FPGA implementation.

The processing chain developed in Simulink consists of FIR filtering, difference computation, absolute-value processing, gain application, threshold comparison, and event validation. The final output of the processing chain is the detection signal, denoted by *Detect*.

An additional parameter-selection study was conducted before fixing the final FPGA configuration. Two parameters were investigated systematically: the amplification gain and the detection threshold. Their influence was evaluated through comparative simulations in order to identify a practical operating point that provides sufficient detection sensitivity without excessive triggering.

3.2 Input Signal Generation

The reference model operates on a sampled digital signal representative of the electrical response obtained from a cable-harness measurement.

The dataset used for the digital processing is associated with a sampling frequency of 1MHz, corresponding to a sampling period of

$$T_s = \frac{1}{1 \times 10^6} = 1. \quad (3.1)$$

The parameter-selection experiments were performed using a simulation stop time of 0.1s. This common simulation window was maintained for all gain and threshold configurations in order to make the observed behaviors directly comparable.

The input signal is processed sequentially by the digital processing chain. The signal itself is not directly compared with the detection threshold. Instead, the detection decision

is obtained after filtering and variation extraction.

The principal processing sequence is therefore

$$x[n] \rightarrow y[n] \rightarrow d[n] \rightarrow |d[n]| \rightarrow G|d[n]| \rightarrow Detect, \quad (3.2)$$

where $x[n]$ denotes the input signal, $y[n]$ the filtered signal, $d[n]$ the signal difference, and G the amplification gain.

3.3 Fault Scenarios

The reference simulations include signal variations representative of abnormal cable-harness behavior. These variations are processed by the digital detection chain and are used to evaluate the response of the detector.

The objective of the simulation study is not to claim that every detected pulse corresponds to a physical fault in a manufactured cable harness. Instead, the simulations are used to evaluate the behavior of the proposed digital decision mechanism under controlled signal conditions.

This distinction is important when interpreting the parameter-selection results. The waveforms demonstrate the sensitivity and triggering behavior of the algorithm, whereas a rigorous evaluation of detection accuracy would require known fault-injection times and corresponding ground-truth labels.

The parameter study therefore focuses primarily on the relative behavior of the detector for different gain and threshold configurations.

3.4 Fixed-Point Representation

The FPGA-oriented signal representation uses the signed fixed-point format

$$\text{fixdt}(1,16,12). \quad (3.3)$$

This format corresponds to a signed 16-bit representation with 12 fractional bits. The associated scaling factor is

$$S = 2^{12} = 4096. \quad (3.4)$$

The numerical value represented by an integer code N can therefore be expressed as

$$x = \frac{N}{4096}. \quad (3.5)$$

The representable numerical range is approximately

$$-8 \leq x < 8. \quad (3.6)$$

This fixed-point representation was retained throughout the reference model and FPGA-oriented implementation in order to maintain a consistent numerical representation between the algorithmic and hardware domains.

The fixed-point representation is particularly relevant for FPGA implementation because the word length and fractional precision are explicitly defined. This makes the numerical behavior more predictable and avoids the additional hardware cost associated with a floating-point implementation.

3.5 Digital Filtering

The first processing stage is a Finite Impulse Response (FIR) filter.

For an input sequence $x[n]$, the filtered output can generally be expressed as

$$y[n] = \sum_{k=0}^{M-1} h[k]x[n-k], \quad (3.7)$$

where $h[k]$ denotes the FIR coefficients and M is the filter length.

The FIR stage performs the initial signal conditioning in the digital domain. Its output is subsequently used for variation detection.

An important observation during the parameter-selection experiments is that changing the gain or threshold does not modify the FIR operation itself. Consequently, the FIR output remains qualitatively similar when the gain or threshold is changed, whereas the downstream Slope and Detect signals change significantly.

This behavior is consistent with the structure of the processing chain because the gain and threshold are applied after the filtering stage.

3.6 Difference Computation

The filtered signal is subsequently processed by a difference operation. The difference emphasizes variations between successive processed samples and can be represented as

$$d[n] = y[n] - y[n-1]. \quad (3.8)$$

A relatively stable signal produces small values of $d[n]$, whereas an abrupt variation produces a larger magnitude.

The difference operation therefore provides the signal quantity on which the subsequent detection mechanism is based.

3.7 Absolute-Value Processing

Because both positive and negative variations may be relevant to fault detection, the magnitude of the difference is calculated:

$$a[n] = |d[n]|. \quad (3.9)$$

The use of the absolute value makes the detector independent of the direction of the signal transition. A positive and a negative variation of similar magnitude therefore produce comparable detection quantities.

3.8 Gain Selection

The absolute difference is amplified by a configurable gain G :

$$s[n] = Ga[n]. \quad (3.10)$$

The resulting signal is referred to as the *Slope* signal in the Simulink model.

The gain determines the amplitude presented to the threshold comparator. A small gain may prevent meaningful variations from reaching the threshold, whereas an excessively large gain may cause smaller signal fluctuations to trigger the detector.

Five gain values were investigated:

$$G \in \{10, 25, 50, 100, 1000\}. \quad (3.11)$$

For this comparison, the threshold was maintained at the reference value used for the study, while the simulation stop time was fixed at 0.1s.

3.8.1 Case 1: Gain = 10

For $G = 10$, the Slope signal exhibits a relatively small amplitude, approximately within the range of 0–2 in the observed simulation.

The Detect signal remains at zero throughout the considered simulation interval. Consequently, no distinct detection pulses are produced.

This behavior indicates that the amplification is insufficient to generate threshold crossings under the selected threshold. The configuration is therefore conservative, with very low triggering activity, but its sensitivity is too low for the considered signal conditions.

It should be emphasized that a lower gain is not intrinsically invalid because the threshold could also be reduced. However, under the common threshold used in this comparison, $G = 10$ does not provide a useful operating point.

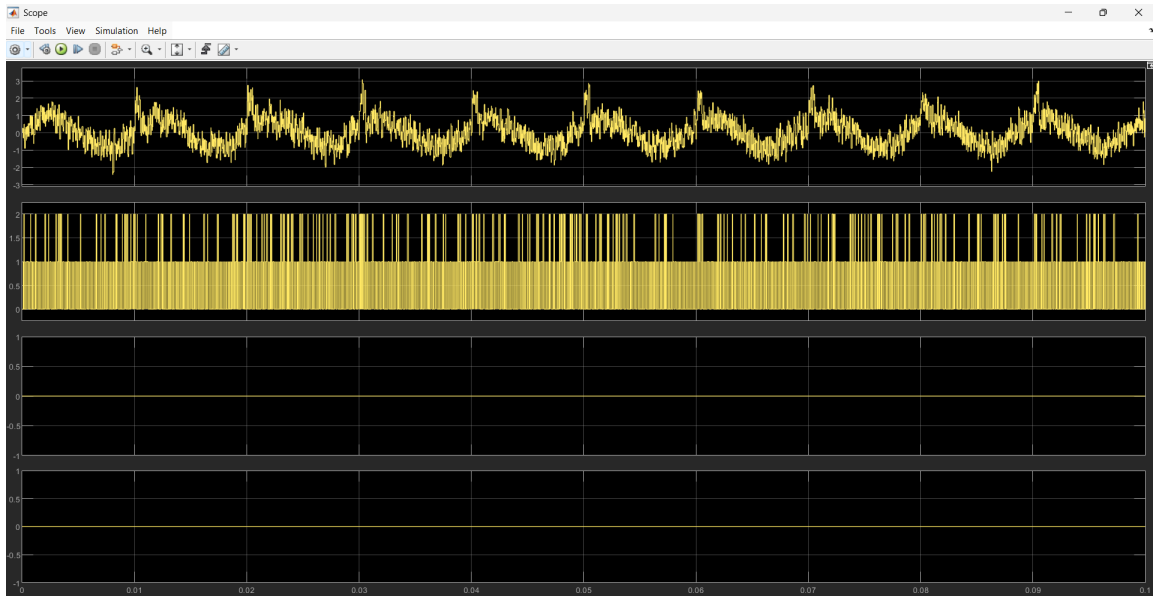


Figure 3.1: MATLAB/Simulink simulation for $G = 10$.

3.8.2 Case 2: Gain = 25

When the gain is increased to $G = 25$, the amplitude of the Slope signal increases, reaching approximately 5 in the observed simulation.

Despite this improvement, the Detect signal remains at zero. The amplified variations still do not produce sufficient threshold crossings to generate distinct detection events.

Compared with $G = 10$, this configuration provides greater sensitivity while maintaining low triggering activity. Nevertheless, it remains below the desired operating region for the selected threshold because the detector does not generate visible detection events.

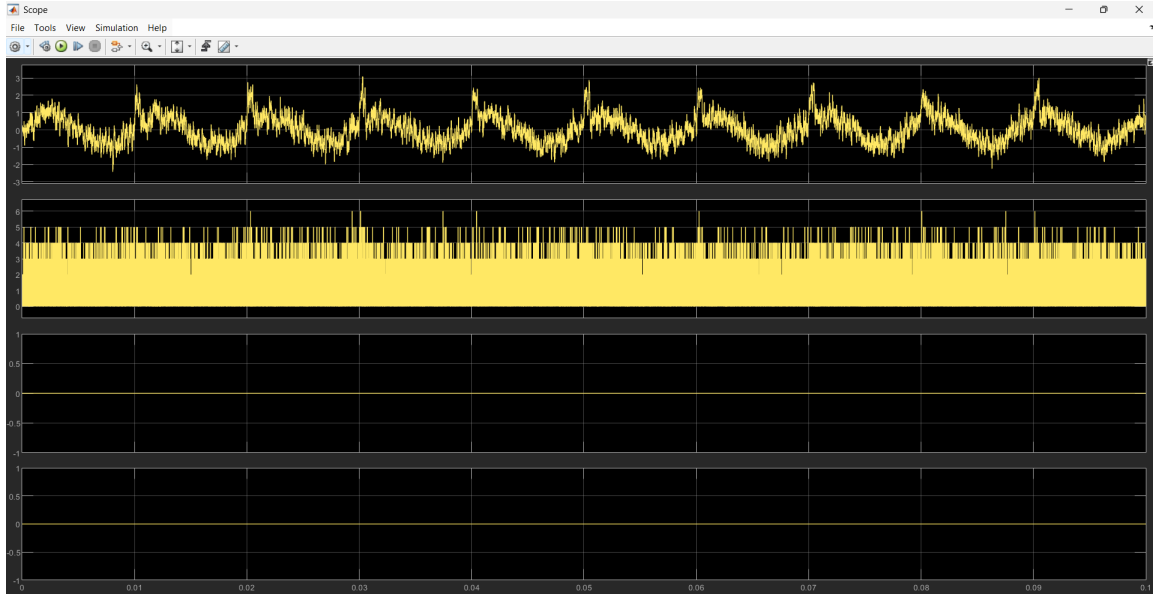


Figure 3.2: MATLAB/Simulink simulation for $G = 25$.

3.8.3 Case 3: Gain = 50

For $G = 50$, the Slope signal reaches approximately 10–12 for the largest observed variations.

In contrast with the lower-gain configurations, the Detect signal now exhibits distinct and separated detection pulses. In the considered simulation, visible detection events occur approximately around

$$0.03, \quad 0.06, \quad 0.08, \quad 0.09 \text{ s.} \quad (3.12)$$

The detection signal is not permanently asserted. Instead, the detector produces discrete pulses separated by non-detection intervals. The Timestamp signal also changes in association with these detection events.

The Timestamp behavior discussed in this section refers exclusively to the MATLAB/Simulink reference model and is not part of the implemented FPGA external interface.

This behavior represents a substantially more useful operating point than the $G = 10$ and $G = 25$ cases. The detector is sufficiently sensitive to produce distinct events, while it does not exhibit the excessive continuous triggering observed for higher gain values.

Therefore, under the considered threshold and simulation conditions, $G = 50$ provides the best observed compromise between sensitivity and selectivity among the tested gain values.

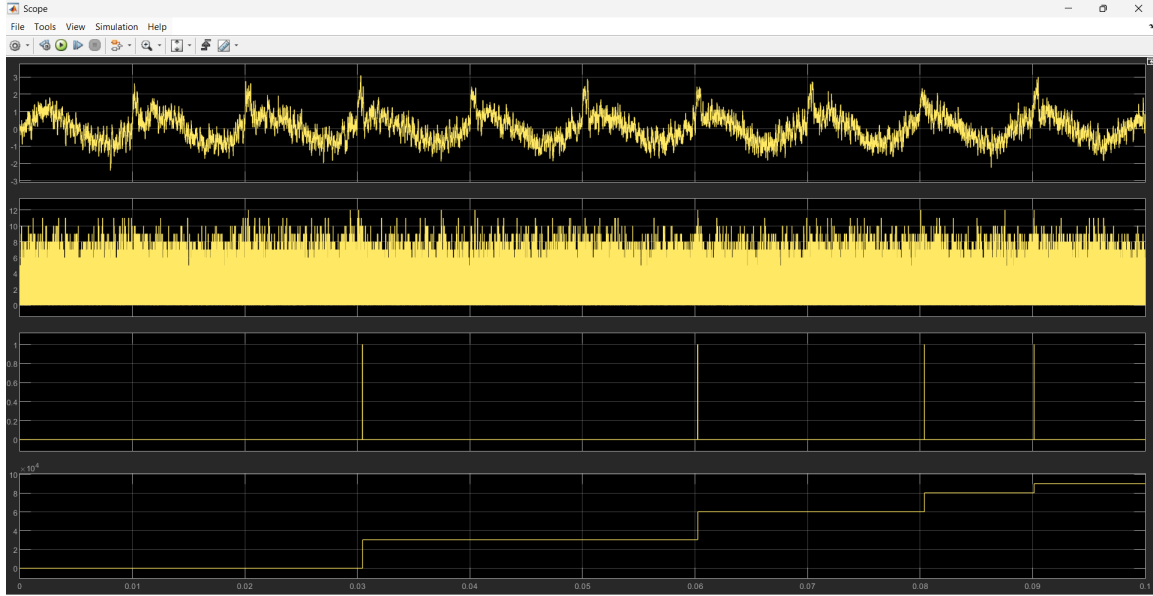


Figure 3.3: MATLAB/Simulink simulation for $G = 50$.

3.8.4 Case 4: Gain = 100

For $G = 100$, the Slope amplitude increases significantly and reaches approximately 25 in the observed simulation.

The Detect signal becomes active for a large portion of the simulation, with only intermittent low intervals. The Timestamp therefore changes much more frequently than in the $G = 50$ case.

The increased gain improves sensitivity, but this improvement is accompanied by excessive triggering. Smaller variations that remained below the decision level at $G = 50$ are more likely to cross the threshold after amplification.

This behavior indicates a loss of selectivity. The detector becomes overly sensitive to variations that should not necessarily generate distinct detection events.

Consequently, $G = 100$ is considered excessive under the tested conditions.

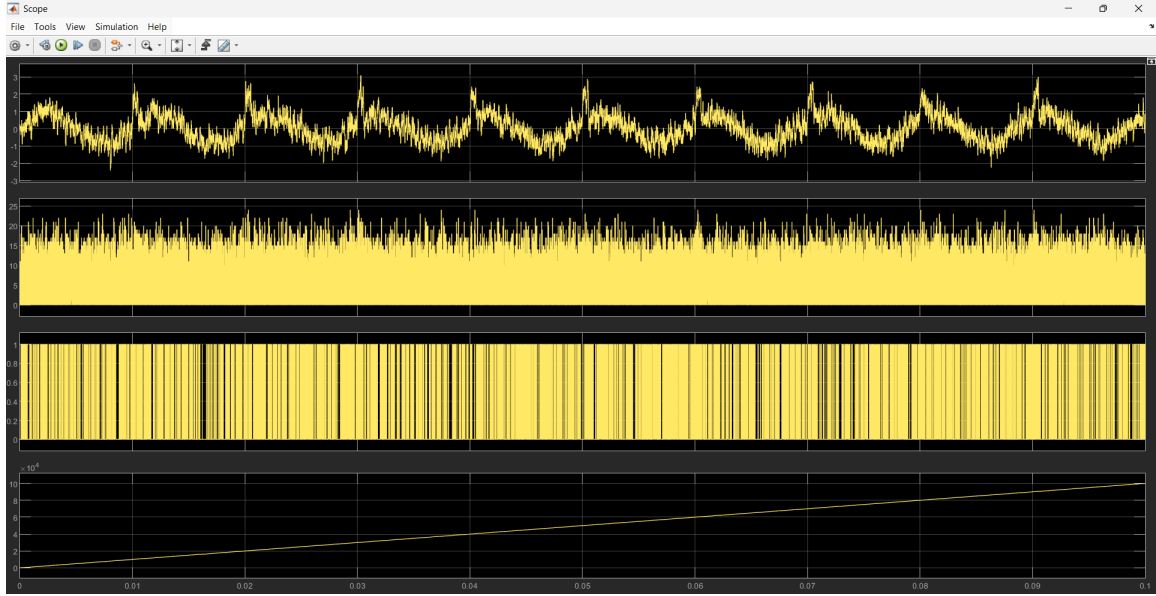


Figure 3.4: MATLAB/Simulink simulation for $G = 100$.

3.8.5 Case 5: Gain = 1000

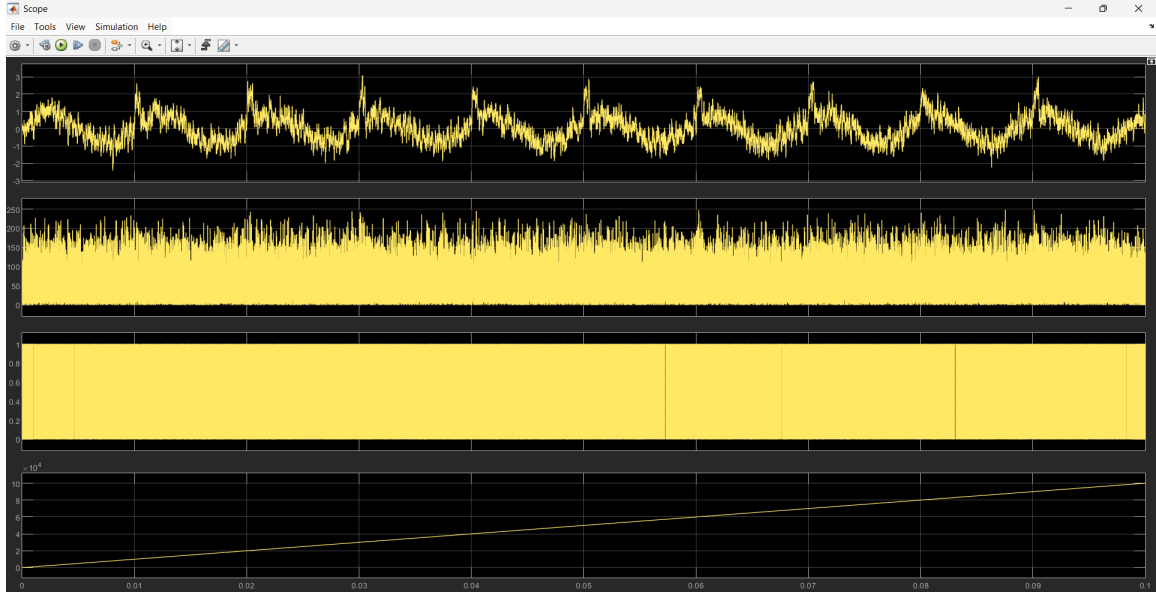
The configuration $G = 1000$ represents an extreme amplification case. The Slope signal reaches values on the order of approximately 250 in the observed simulation.

The Detect signal becomes practically continuously asserted, with only limited interruptions. The Timestamp similarly changes almost continuously.

This configuration clearly demonstrates the negative effect of excessive amplification. The threshold comparator becomes sensitive to very small signal variations, resulting in a substantial loss of selectivity.

Furthermore, such a large gain would require careful analysis of numerical range, intermediate word lengths, and saturation behavior before being considered for a fixed-point hardware implementation.

The $G = 1000$ configuration is therefore clearly unsuitable for the considered detection conditions.

Figure 3.5: MATLAB/Simulink simulation for $G = 1000$.

3.8.6 Gain Comparison

The five simulations reveal a clear qualitative relationship between gain and detector behavior.

Table 3.1: Qualitative comparison of the tested gain values.

Gain	Slope	Detect	Sensitivity	Triggering	Assessment
10	$\sim 0-2$	0	Low	Very low	Too insensitive
25	$\sim 0-5$	0	Low-moderate	Low	Still below threshold
50	$\sim 0-12$	Distinct pulses	Moderate-high	Controlled	Best observed compromise
100	$\sim 0-25$	Almost continuously 1	Very high	Excessive	Too sensitive
1000	$\sim 0-250$	Almost continuously 1	Excessive	Very high	Clearly unsuitable

The observed behavior can be summarized by

$$G \uparrow \implies s[n] \uparrow \implies \text{threshold crossings increase.} \quad (3.13)$$

The objective is therefore not to maximize the gain, but to select an amplification level that produces meaningful detection events without excessive triggering.

Among the five tested configurations, $G = 50$ provides the most balanced behavior. The lower gains fail to produce visible detection events under the considered threshold, whereas the higher gains produce excessive detection activity.

The value $G = 50$ is therefore selected for the subsequent threshold study.

3.9 Threshold Selection

After selecting $G = 50$, the detection threshold was investigated independently.

The threshold comparator is defined as

$$q[n] = \begin{cases} 1, & s[n] > T, \\ 0, & s[n] \leq T, \end{cases} \quad (3.14)$$

where T is the threshold.

Five threshold values were evaluated:

$$T \in \{5, 7.5, 9.5, 12, 15\}. \quad (3.15)$$

The gain was fixed at $G = 50$ for all threshold simulations, and the simulation stop time remained 0.1s.

3.9.1 Case 1: Threshold = 5

With $T = 5$, the threshold is relatively low compared with the observed Slope amplitudes.

The Slope signal frequently reaches values between approximately 6 and 10, meaning that many signal variations exceed the decision level.

Consequently, the Detect signal becomes active for a large fraction of the simulation and can appear almost continuously asserted. The Timestamp also changes frequently.

This configuration provides high sensitivity, but the detector becomes too permissive. Smaller variations are more likely to cross the threshold, resulting in a high apparent risk of unwanted triggering.

Therefore, although $T = 5$ is highly sensitive, its selectivity is insufficient for the considered operating conditions.

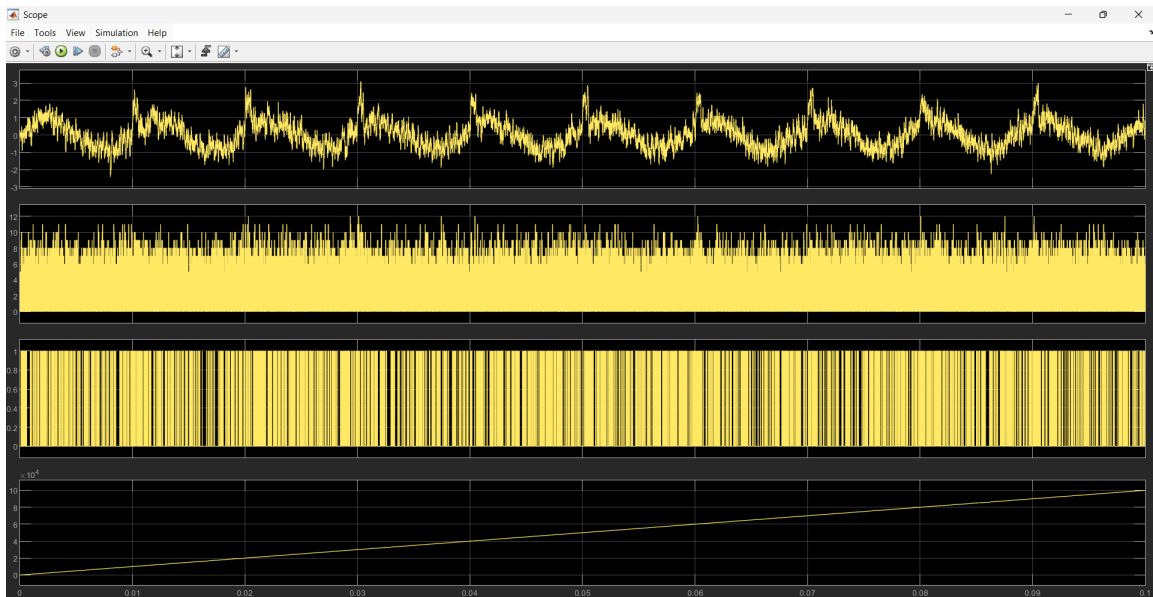


Figure 3.6: MATLAB/Simulink simulation for $T = 5$ with $G = 50$.

3.9.2 Case 2: Threshold = 7.5

Increasing the threshold to $T = 7.5$ reduces the detection activity compared with the $T = 5$ case.

The Detect signal is no longer continuously high, but numerous detection pulses are still visible. The Timestamp consequently exhibits several changes associated with these events.

This configuration represents an improvement because more small variations are rejected. However, the detector remains relatively sensitive and continues to produce a comparatively large number of detection events.

Thus, $T = 7.5$ provides better selectivity than $T = 5$, but it does not yet provide the best observed balance.

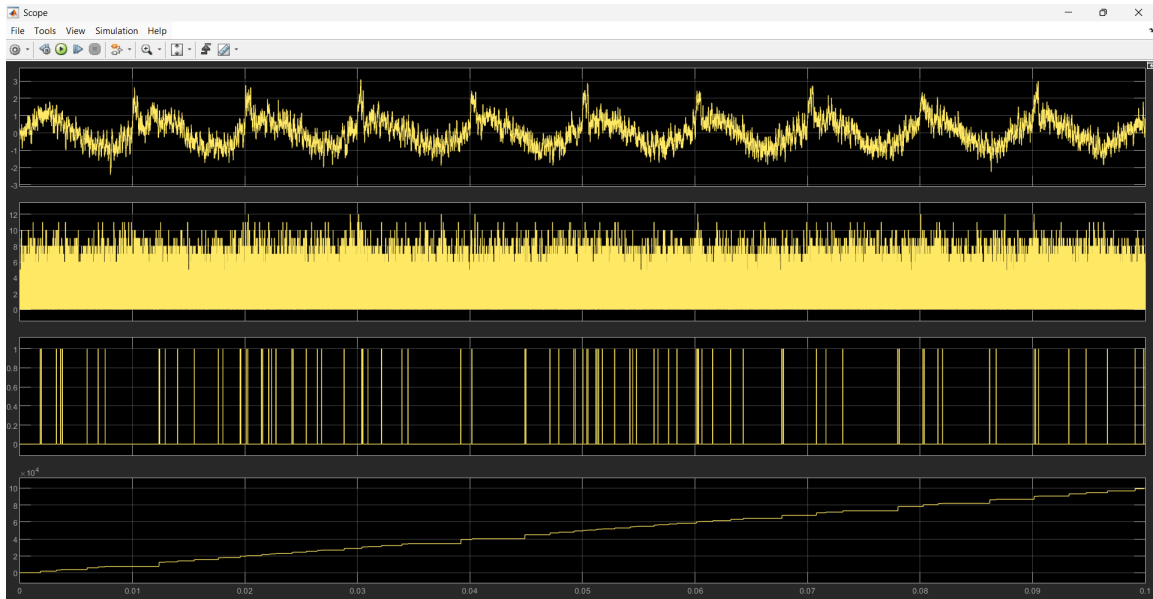


Figure 3.7: MATLAB/Simulink simulation for $T = 7.5$ with $G = 50$.

3.9.3 Case 3: Threshold = 9.5

For $T = 9.5$, the Detect signal exhibits distinct and separated detection pulses. In the considered simulation, visible events occur approximately around

$$0.03, \quad 0.06, \quad 0.08, \quad 0.09 \text{ s.} \quad (3.16)$$

The Timestamp changes correspondingly rather than progressing continuously.

At the same time, smaller fluctuations of the Slope signal that remain below the threshold do not produce visible detection events. The resulting behavior is therefore substantially more selective than that obtained with thresholds of 5 and 7.5.

The threshold is sufficiently high to reject many smaller variations while remaining below the larger excursions generated under the selected gain.

This configuration provides the clearest separation between detection and non-detection intervals among the tested threshold values.

It should nevertheless be emphasized that the available waveforms alone cannot establish whether every detected pulse corresponds to a true physical cable fault. Such a conclusion would require known fault-injection times and ground-truth labels.

Within the tested simulation conditions, however, $T = 9.5$ provides the best observed compromise between sensitivity and selectivity.

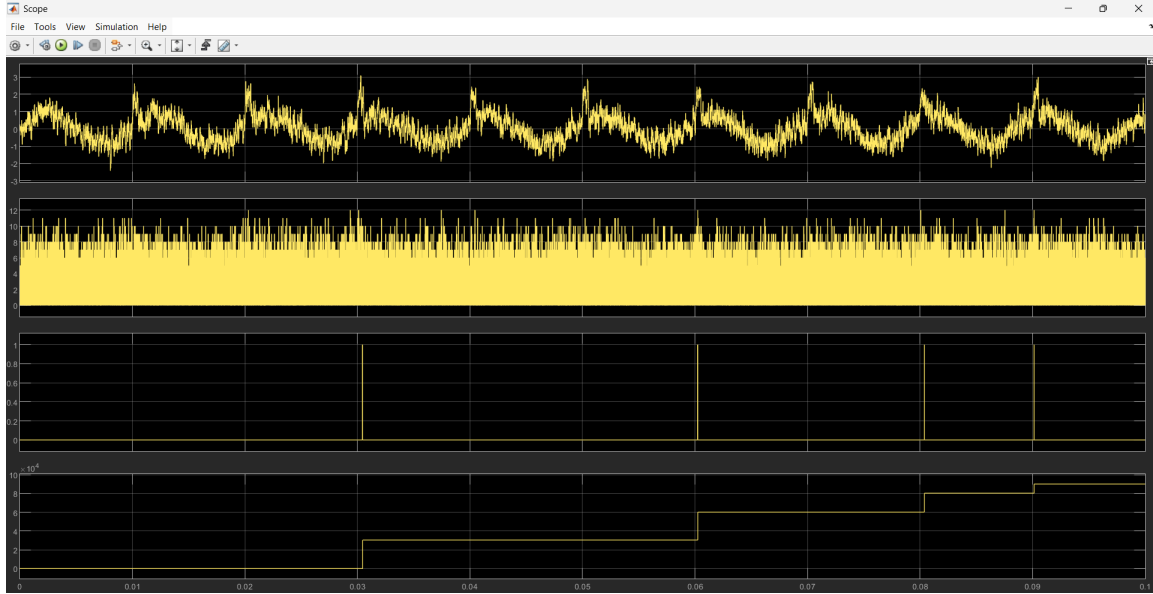


Figure 3.8: MATLAB/Simulink simulation for $T = 9.5$ with $G = 50$.

3.9.4 Case 4: Threshold = 12

For $T = 12$, no visible Detect pulse is observed in the considered simulation. The Timestamp remains essentially constant.

This indicates that the threshold has become higher than the principal Slope excursions generated under the selected gain and signal conditions.

The detector therefore becomes excessively conservative. Although this configuration strongly reduces unwanted triggering, it also suppresses the detection events that were observed at lower threshold values.

Consequently, $T = 12$ is considered too high for the current gain and signal conditions.

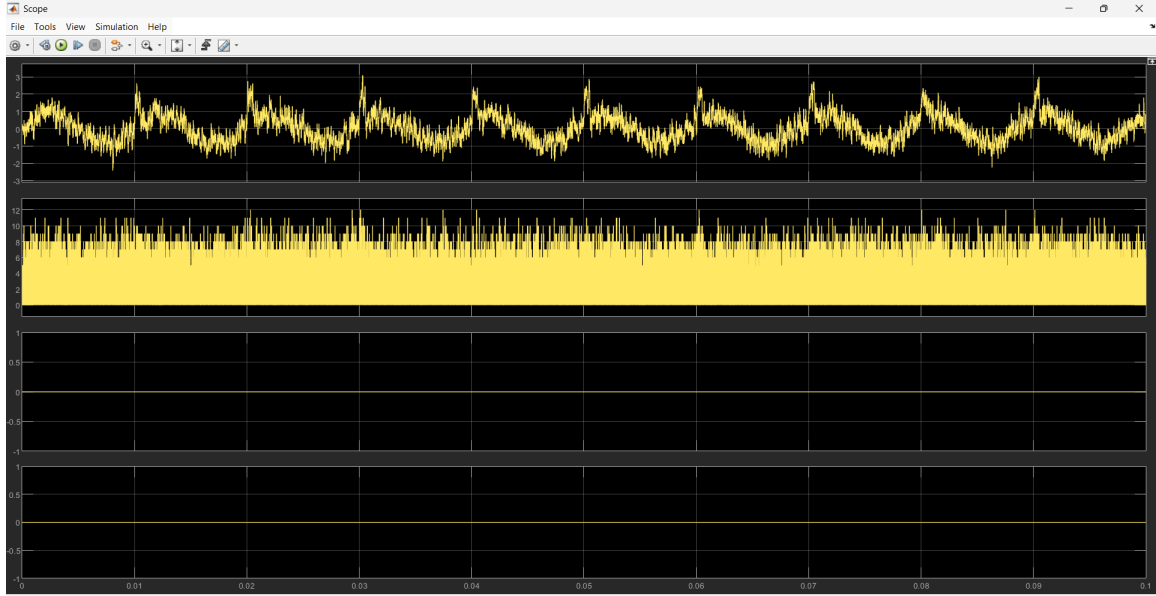


Figure 3.9: MATLAB/Simulink simulation for $T = 12$ with $G = 50$.

3.9.5 Case 5: Threshold = 15

The threshold $T = 15$ further increases the decision level.

As observed in the simulation, the Detect signal remains at zero and the Timestamp remains constant. The detector therefore does not respond to the signal excursions that produced detection events for lower threshold values.

This configuration provides strong rejection of small variations, but the resulting sensitivity is insufficient.

The $T = 15$ case is consequently rejected as excessively conservative.

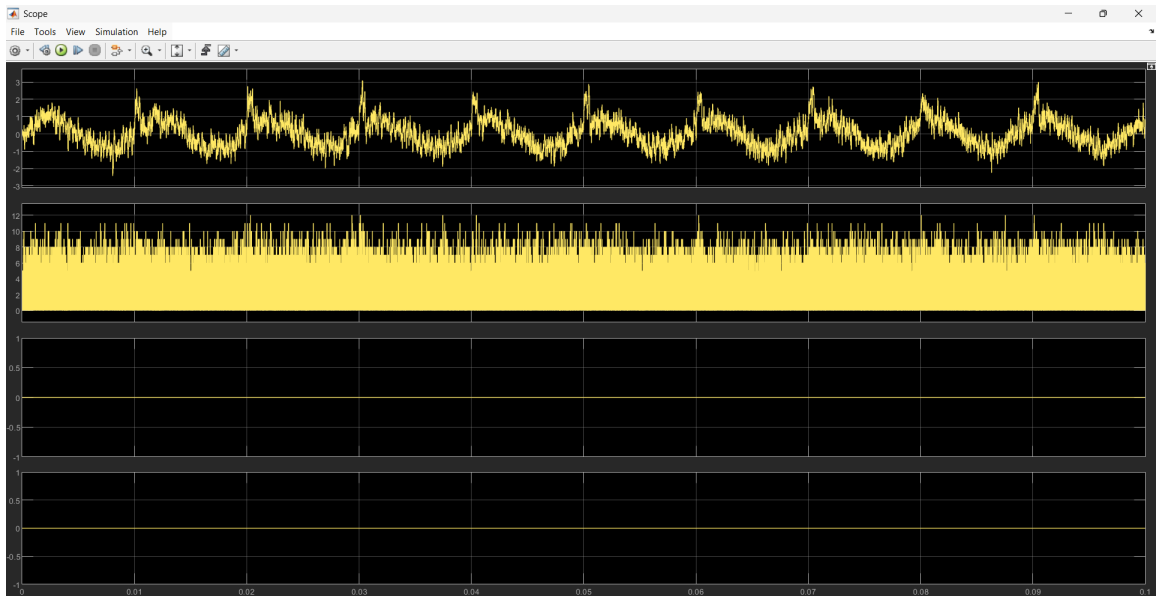


Figure 3.10: MATLAB/Simulink simulation for $T = 15$ with $G = 50$.

3.9.6 Threshold Comparison

The five simulations demonstrate the expected trade-off between detection sensitivity and selectivity.

Table 3.2: Qualitative comparison of the tested threshold values.

Threshold	Detect behavior	Timestamp	Sensitivity	Assessment
5	Almost continuously active	Nearly continuous	Very high	Excessively sensitive
7.5	Numerous pulses	Numerous steps	High	Still relative sensitive
9.5	Distinct separated pulses	Event-associated steps	Moderate-high	Best observed compromise
12	No visible detection	Constant	Very low	Too high
15	No visible detection	Constant	Very low	Clearly too high

The general behavior can be summarized as

$$T \uparrow \implies \text{threshold crossings} \downarrow. \quad (3.17)$$

A low threshold therefore increases sensitivity but also increases the apparent triggering activity. Conversely, a high threshold reduces triggering but increases the risk of missing relevant signal variations.

The tested values consequently define three qualitative operating regions:

1. **Low-threshold region:** $T = 5$, characterized by excessive triggering and poor selectivity.
2. **Intermediate-threshold region:** $T = 7.5$ and $T = 9.5$, where the detector remains responsive while providing increasingly better selectivity.
3. **High-threshold region:** $T = 12$ and $T = 15$, where the detector becomes excessively conservative and no longer generates visible detection events.

Among the tested values, $T = 9.5$ lies in the most useful operating region.

3.10 Final Parameter Selection

The gain and threshold are coupled design parameters because the quantity presented to the threshold comparator is

$$s[n] = G|d[n]|. \quad (3.18)$$

Consequently, increasing the gain increases the amplitude of the signal presented to the comparator, while increasing the threshold raises the minimum amplitude required to generate a detection event.

The parameter-selection procedure was therefore performed in two stages.

First, the gain was investigated while maintaining a common threshold. The results showed that gains of 10 and 25 produced insufficient detection activity, whereas gains of

100 and 1000 produced excessive triggering. The intermediate value $G = 50$ generated distinct detection pulses while avoiding the nearly continuous detection observed at higher gains.

Second, with $G = 50$ fixed, the threshold was varied from 5 to 15. Thresholds of 5 and 7.5 produced frequent detections, while thresholds of 12 and 15 suppressed the detection response. The value $T = 9.5$ produced distinct and separated detection events while rejecting a substantial proportion of smaller signal variations.

The selected configuration is therefore

$$\boxed{G = 50, \quad T = 9.5}. \quad (3.19)$$

The selection is supported by the comparative simulation results rather than by an arbitrary parameter choice. In particular, it avoids the two undesirable extremes observed during the study:

- **Under-detection:** insufficient threshold crossings for low gains or excessively high thresholds.
- **Over-detection:** excessive threshold crossings for high gains or excessively low thresholds.

The selected values provide the best observed compromise among the configurations tested under the considered simulation conditions.

It is important to state that $G = 50$ and $T = 9.5$ are not claimed to be globally optimal parameters. They are the **best observed configuration among the tested values under the considered signal scenarios and simulation conditions**.

A rigorous global optimization would require a larger labeled dataset containing both fault and non-fault cases, together with quantitative metrics such as true positives, false positives, false negatives, detection rate, precision, recall, and detection latency.

Nevertheless, the parameter sweep provides a rational engineering basis for fixing the configuration used in the subsequent FPGA implementation and verification stages.

3.11 Reference Model Summary

The MATLAB/Simulink reference model establishes the functional baseline for the FPGA implementation.

The complete digital processing sequence is

$$x[n] \rightarrow \text{FIR} \rightarrow \Delta \rightarrow |\Delta| \rightarrow G \rightarrow T \rightarrow \text{Event Validation} \rightarrow \text{Detect}. \quad (3.20)$$

The parameter study demonstrated that both amplification and threshold selection strongly influence the detector behavior. The gain determines the amplitude of the processed variations presented to the comparator, while the threshold determines the minimum variation required to generate a detection event.

Based on the comparative simulations, the configuration

$$\boxed{G = 50, \quad T = 9.5} \quad (3.21)$$

was selected as the reference configuration for the subsequent HDL generation, FPGA implementation, and functional verification.

This configuration is therefore carried forward to Chapter 4, where the corresponding hardware architecture and VHDL implementation are described. ““

Chapter 4

FPGA Architecture and HDL Implementation

4.1 Target FPGA Device

The proposed real-time processing system was implemented for an AMD/Xilinx Artix-7 FPGA. The selected device is the `xc7a100tcsg324-1`, belonging to the Artix-7 XC7A100T family.

The choice of this device provides sufficient digital resources for the implemented signal-processing architecture while remaining representative of a practical embedded FPGA platform. The implementation was developed using MATLAB/Simulink and HDL Coder for hardware description generation, followed by synthesis, implementation, and timing analysis in AMD Vivado.

The main implementation parameters are summarized in Table 4.1.

Table 4.1: Main FPGA implementation parameters.

Parameter	Value
FPGA family	AMD/Xilinx Artix-7
Target device	XC7A100T
Exact part	<code>xc7a100tcsg324-1</code>
System clock	100 MHz
Processing clock	40 MHz
Processing period	25 ns
Data format	Signed fixed-point
Word length	16 bits
Fractional bits	12
Fixed-point format	<code>fixdt(1,16,12)</code>

The processing clock of 40 MHz corresponds to a clock period of 25 ns. This clock is used by the generated processing logic and therefore defines the main synchronous operating rate of the implemented detection chain.

4.2 MATLAB/Simulink to VHDL Generation Flow

The FPGA implementation follows a model-based hardware design methodology. MATLAB/Simulink was first used to develop and validate the digital signal-processing algorithm. Once the reference model and its parameters were established, HDL Coder was used to generate synthesizable VHDL code.

The resulting VHDL design was then integrated into an FPGA-oriented project and processed through the Vivado synthesis and implementation flow.

The complete development chain is therefore:

**MATLAB/Simulink → HDL Coder → VHDL → Vivado → FPGA
implementation**

This methodology provides a direct connection between the algorithmic reference model and the hardware implementation. It also allows the same processing architecture to be examined at different abstraction levels, from signal-processing behavior in Simulink to RTL simulation and FPGA implementation results.

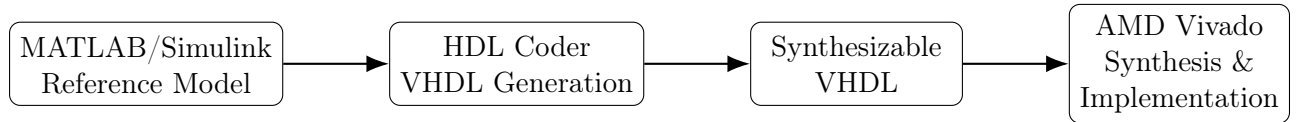


Figure 4.1: Model-based FPGA development flow from MATLAB/Simulink to VHDL generation and Vivado implementation.

4.3 Clock Architecture

The design uses a 100 MHz input/system clock and generates a 40 MHz processing clock using the FPGA clock-management resources.

The clock-generation stage is implemented using the Vivado-generated `clk_wiz_0` module. *The 100 MHz input clock management circuitry, which generates the 40 MHz clock used by the processing logic.*

The resulting processing period is:

$$T_{\text{clk}} = \frac{1}{40 \text{ MHz}} = 25 \text{ ns.} \quad (4.1)$$

The generated processing clock drives the synchronous FPGA processing elements. This provides a deterministic timing reference for the FIR filter, arithmetic operations, threshold comparison, and event-validation logic.



Figure 4.2: FPGA clock architecture showing the 100 MHz input clock and generated 40 MHz processing clock.

4.4 Fixed-Point Hardware Representation

The signal-processing architecture uses signed fixed-point arithmetic in order to obtain a hardware-efficient representation of the input and intermediate digital signals.

The selected format is:

$$\text{fixdt}(1, 16, 12), \quad (4.2)$$

where the first parameter indicates a signed representation, the second parameter specifies a total word length of 16 bits, and the third parameter specifies 12 fractional bits.

The corresponding scaling factor is therefore:

$$2^{12} = 4096. \quad (4.3)$$

A real-valued quantity x is represented in the fixed-point domain according to:

$$x_{\text{fixed}} \approx x_{\text{real}} \times 4096. \quad (4.4)$$

For a signed 16-bit representation with 12 fractional bits, the nominal numerical range is approximately:

$$-8 \leq x < 8. \quad (4.5)$$

For example, the hexadecimal values 0000, 7FFF, and 8000 correspond respectively to approximately 0, +7.9998, and -8.0000 in the selected fixed-point representation.

The use of fixed-point arithmetic is important for FPGA implementation because it avoids the hardware cost associated with floating-point processing while providing sufficient numerical resolution for the considered detection algorithm.

4.5 Real-Time Signal-Processing Chain

The implemented FPGA processing chain follows the architecture established in the MATLAB/Simulink reference model. The main processing stages are:

1. FIR filtering;
2. difference computation;
3. absolute-value computation;
4. gain application;
5. threshold comparison;
6. event validation;
7. generation of the external detection output.

The processing chain can be represented conceptually as:

Input $\rightarrow$ FIR $\rightarrow$ Difference $\rightarrow$ Absolute Value $\rightarrow$ Gain $\rightarrow$ Threshold $\rightarrow$ Event
Validation $\rightarrow$ Detect

The selected parameters obtained from the analysis presented in Chapter 3 are used in the hardware implementation. In particular, the implemented gain is $G = 50$ and the selected threshold is $T = 9.5$.

4.5.1 FIR Filtering

The first processing stage is a finite impulse response (FIR) filter. Its purpose is to condition the sampled signal before the fault-related variation is evaluated.

For an input sequence $x[n]$, the FIR operation can be expressed as:

$$y[n] = \sum_{k=0}^{M-1} h[k]x[n-k], \quad (4.6)$$

where $h[k]$ represents the filter coefficients and M is the number of filter coefficients.

The FIR stage provides a controlled digital preprocessing step and reduces the influence of high-frequency variations before the subsequent difference and detection operations.

4.5.2 Difference Computation

After filtering, the variation of the signal is evaluated through a difference operation. The resulting signal represents the local change of the filtered waveform.

It can be expressed conceptually as:

$$d[n] = y[n] - y[n-1]. \quad (4.7)$$

This operation emphasizes transitions and rapid changes in the processed signal, which are relevant to the considered fault-detection strategy.

4.5.3 Absolute-Value Operation

Because the detection mechanism is intended to respond to the magnitude of the variation rather than its polarity, the absolute value of the difference is calculated:

$$a[n] = |d[n]|. \quad (4.8)$$

This allows both positive and negative signal transitions to contribute to the same detection criterion.

4.5.4 Gain Application

The absolute difference is then multiplied by the selected gain:

$$s[n] = G a[n]. \quad (4.9)$$

For the implemented configuration:

$$G = 50. \quad (4.10)$$

The gain increases the numerical separation between small variations and larger signal transitions, making the subsequent threshold comparison more suitable for the selected fixed-point representation and detection criterion.

4.5.5 Threshold Comparison

The scaled signal is compared with the selected threshold:

$$T = 9.5. \quad (4.11)$$

The comparison operation can be expressed as:

$$q[n] = \begin{cases} 1, & s[n] > T, \\ 0, & s[n] \leq T. \end{cases} \quad (4.12)$$

The comparator therefore produces a binary indication whenever the processed variation exceeds the selected threshold.

The strict inequality $s[n] > T$ is used in accordance with the implemented detection criterion.

4.5.6 Event Validation

The threshold comparison is followed by an event-validation stage. The purpose of this stage is to avoid treating an isolated instantaneous threshold crossing as a valid event.

The implemented validation logic requires two consecutive qualifying samples before generating a valid detection event.

Conceptually, if $q[n]$ denotes the threshold-comparison result, a valid event requires:

$$q[n-1] = 1 \quad \text{and} \quad q[n] = 1. \quad (4.13)$$

This temporal validation introduces an additional degree of selectivity into the detection process and makes the output dependent not only on the instantaneous signal magnitude but also on its persistence over consecutive processing samples.

4.6 FPGA External Interface

The generated FPGA processing block exposes the input and control signals required for operation together with the final detection output.

The main externally observable signals used during RTL simulation are summarized in Table 4.2.

Table 4.2: Main external FPGA interface signals.

Signal	Width	Function
In1	16 bits	Signed fixed-point input signal
clk	1 bit	Processing clock input
reset	1 bit	Reset control
clk_enable	1 bit	Processing enable control
ce_out	1 bit	Clock-enable output/control
Out3	1 bit	Final detection output

The principal functional output of the implemented FPGA design is Out3, which represents the final detection decision.

The internal signals of the processing chain are not exposed as external FPGA outputs. Consequently, signals such as the FIR output, difference signal, absolute-value output, gain output, comparator state, and event-validation state remain internal to the generated design.

4.7 HDL and Vivado Integration

After HDL generation, the VHDL description was integrated into the Vivado FPGA development environment. Vivado was used for synthesis, implementation, timing analysis, resource estimation, power estimation, and design-rule checking.

The implementation flow consists of the following main steps:

1. HDL generation from the MATLAB/Simulink model;
2. VHDL integration into the FPGA project;
3. clock and interface configuration;
4. RTL synthesis;
5. design implementation and placement/routing;
6. timing analysis;
7. resource-utilization analysis;
8. power estimation;
9. design-rule checking.

The implementation was performed for the xc7a100tcsg324-1 device with a 40 MHz processing-clock requirement.

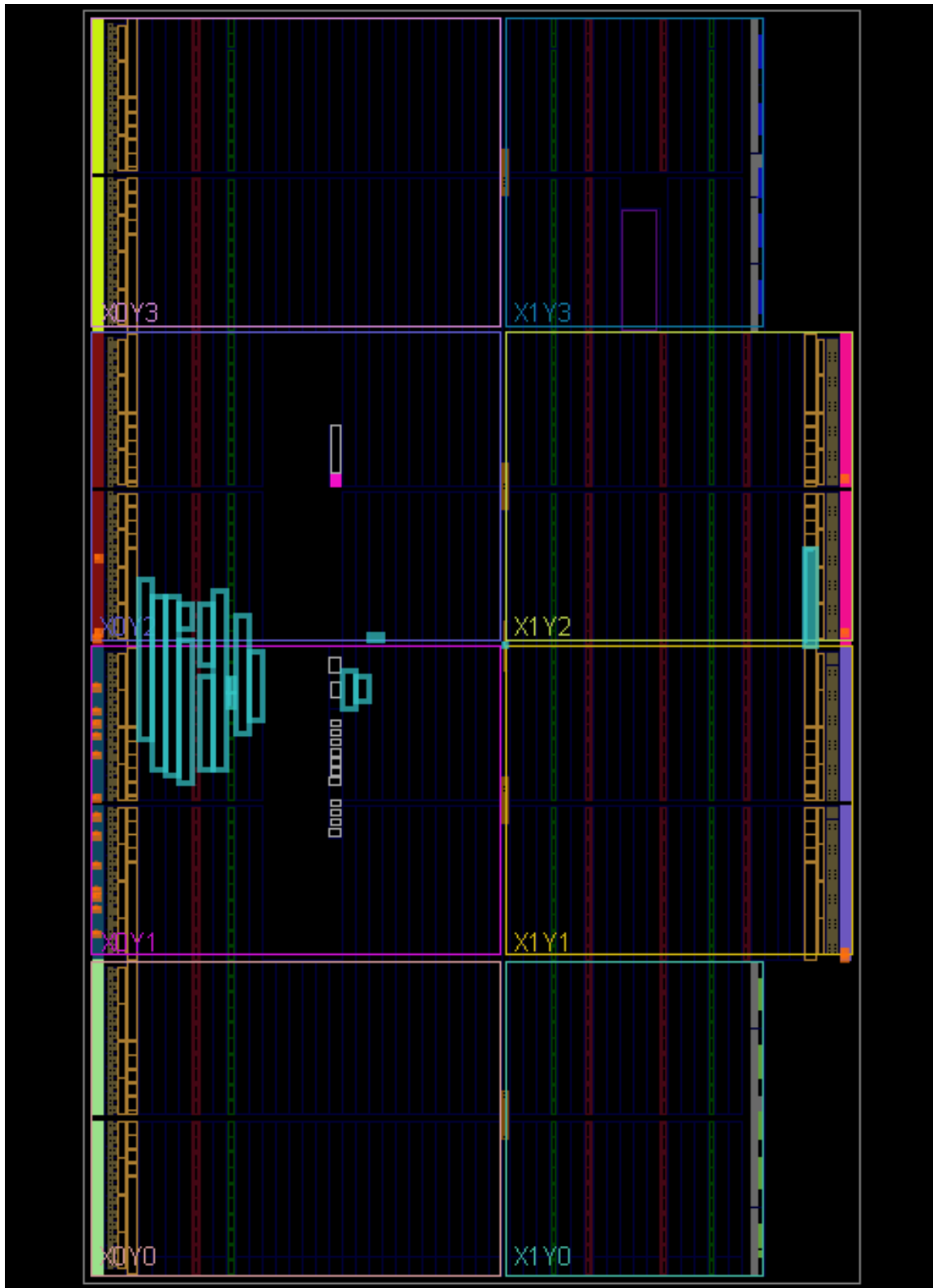


Figure 4.3: Vivado FPGA implementation environment for the generated VHDL design.

4.8 Implementation Configuration Summary

The main configuration parameters used in the final FPGA implementation are summarized in Table 4.3.

Table 4.3: Final FPGA processing configuration.

Parameter	Implemented value
Target FPGA	Artix-7 XC7A100T
Exact device	xc7a100tcsg324-1
System clock	100 MHz
Processing clock	40 MHz
Processing period	25 ns
Input representation	Signed 16-bit fixed-point
Fixed-point format	fixdt(1,16,12)
Gain	50
Threshold	9.5
Event validation	Two consecutive qualifying samples
External detection output	Out3
HDL	VHDL
Implementation tool	AMD Vivado

4.9 Chapter Summary

This chapter described the transformation of the MATLAB/Simulink reference model into a synthesizable FPGA implementation. The design targets an AMD/Xilinx Artix-7 XC7A100T FPGA and uses a 40 MHz processing clock derived from a 100 MHz input clock.

The implemented architecture uses signed 16-bit fixed-point arithmetic and follows the processing chain established in the reference model: FIR filtering, difference computation, absolute value, gain, threshold comparison, and event validation.

The final detection decision is provided through the external Out3 signal. The generated VHDL design was integrated and implemented using Vivado, providing the basis for the functional verification and hardware implementation analysis presented in the following chapters. ““

““latex

Chapter 5

Functional Verification

5.1 Verification Objectives

The functional verification of the proposed cable harness test bench was performed at two complementary levels. First, the MATLAB/Simulink model was used as a reference model to establish the expected behavior of the signal-processing and detection chain. Second, the generated VHDL design was simulated in Vivado to verify the behavior of the FPGA-oriented implementation.

The main objective of this verification is to determine whether the FPGA implementation preserves the intended detection behavior of the MATLAB/Simulink reference model. Particular attention is given to the final detection signal. In the FPGA implementation, this signal is available at the external output Out3, which corresponds to the Detect signal of the reference model.

The verification focuses on the following aspects:

- behavior of the MATLAB/Simulink reference model;
- behavior of the generated VHDL design in Vivado;
- correspondence between the MATLAB/Simulink Detect signal and the FPGA Out3 output;
- detection behavior under normal operating conditions;
- response to fault-related signal variations;
- behavior under repeated detection events;
- reset and initialization behavior;
- limitations of the performed verification.

The verification presented in this chapter is a functional verification based on the tested operating conditions. It is not intended to constitute an exhaustive formal verification of all possible input sequences.

5.2 MATLAB/Simulink Reference Model

The MATLAB/Simulink model described in Chapter 3 serves as the behavioral reference for the FPGA implementation. The reference processing chain consists of the input signal, FIR filtering, slope detection, absolute-value processing, gain, threshold comparison, validation logic, and the final Detect output.

For the selected configuration, the gain and detection threshold are:

$$G = 50 \quad (5.1)$$

and

$$T = 9.5. \quad (5.2)$$

The corresponding detection condition is expressed as:

$$|d[n]| \times 50 > 9.5, \quad (5.3)$$

where $d[n]$ represents the processed difference signal.

The MATLAB/Simulink simulation provides the expected temporal behavior of the detection system. During normal portions of the input signal, the Detect output remains low. When a sufficiently significant signal variation is detected, the model generates a detection pulse.

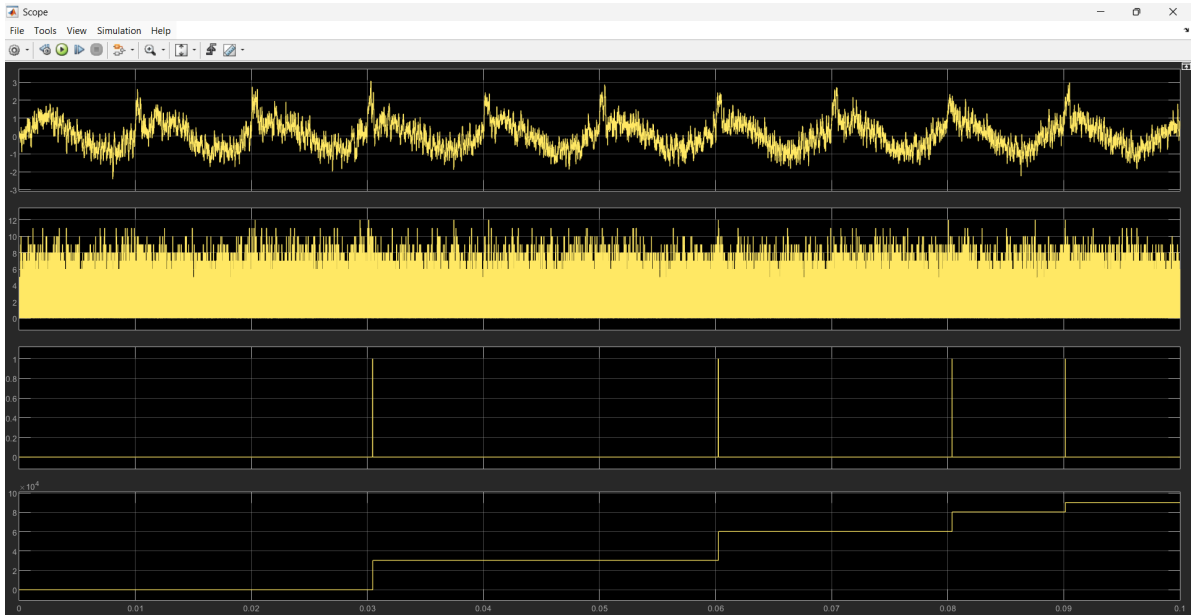


Figure 5.1: MATLAB/Simulink reference simulation showing the input signal, processed signal, detection output, and event timing information.

Figure 5.1 is used as the reference for the subsequent comparison with the FPGA-oriented simulation. The main signal considered for the functional comparison is the Detect output rather than the exact visual shape of every internal processing signal.

The Timestamp signal is used in the MATLAB/Simulink reference model to characterize the timing of detected events. It is considered a reference-model signal and is not used as an FPGA verification output unless an equivalent timestamp implementation is explicitly included in the FPGA design.

5.3 Vivado RTL Simulation

After HDL generation, the design was integrated into the Vivado environment and simulated at the RTL level. The simulation provides access to the externally observable FPGA interface signals, including the clock, reset, input data, control signals, and detection output.

The principal signals considered during the verification are summarized in Table 5.1.

Table 5.1: Main signals considered during RTL functional verification.

Signal	Role	Verification purpose
clk	Processing clock	Synchronization of the RTL design
reset	Reset input	Initialization verification
clk_enable	Clock-enable control	Control of processing activity
In[15:0]	Digital input signal	Input stimulus applied to the FPGA design
Out3	Detection output	Main functional verification output

The Vivado waveform is interpreted primarily through the relationship between the input stimulus and Out3. The internal processing operations generated by HDL Coder are not necessarily exposed as external ports and therefore do not all appear as independent signals in the top-level waveform.

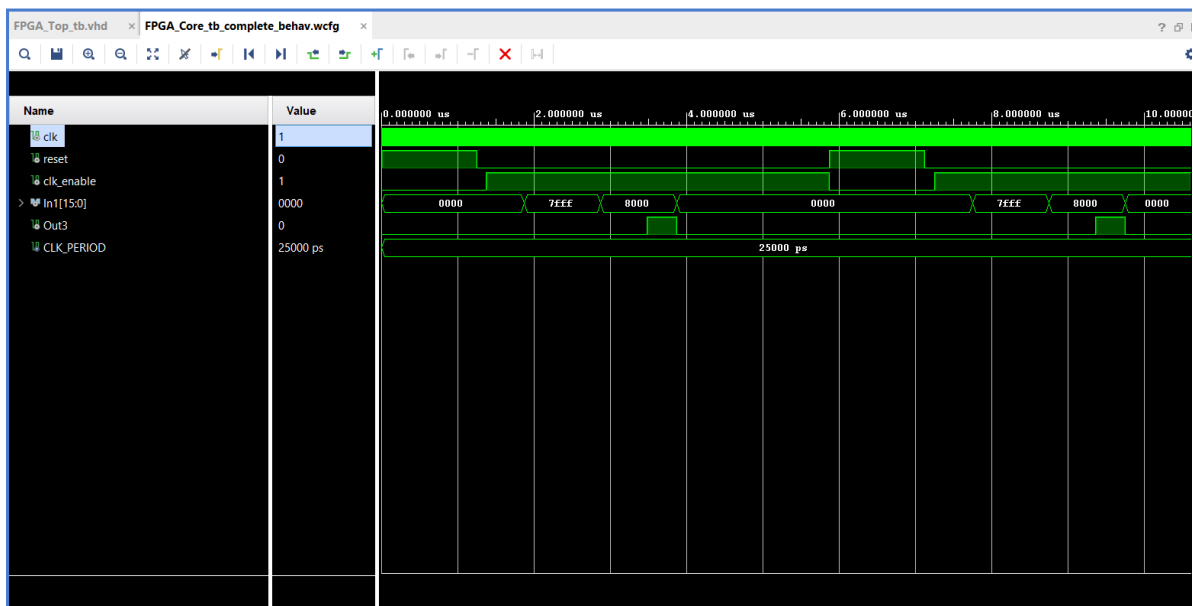


Figure 5.2: Vivado RTL simulation waveform showing the FPGA input, control signals, and the externally observable detection output Out3.

Figure 5.2 provides the RTL-level evidence used to analyze the FPGA detection response. The important observation is that Out3 remains low during normal portions of the test and generates a high pulse when the applied input sequence produces a condition associated with a detected event.

5.4 Comparison Between MATLAB/Simulink and Vivado

The principal functional comparison is established between the Detect signal of the MATLAB/Simulink reference model and the Out3 signal of the Vivado implementation.

Since Out3 is the FPGA output corresponding to Detect, the comparison is based on the occurrence of detection events rather than on a pixel-by-pixel comparison of waveform graphics.

The correspondence is summarized in Table 5.2.

Table 5.2: Functional correspondence between the reference model and FPGA implementation.

Reference model	FPGA implementation	Comparison
Input signal	In[15:0]	Applied input stimulus
FIR processing	HDL implementation	Hardware realization
Slope detection	HDL implementation	Hardware realization
Detect	Out3	Main functional correspondence
Detection event	Detection pulse	Event-level comparison
Normal operation	Out3 = 0	No detection expected
Fault-related event	Out3 = 1	Detection expected

The verification criterion is therefore based on the following principle: events that produce a detection pulse in the MATLAB/Simulink reference should produce the corresponding detection behavior at Out3 in the FPGA-oriented simulation under equivalent input conditions.

This approach is appropriate because the purpose of the FPGA implementation is to reproduce the functional behavior of the reference detection algorithm in a synthesizable hardware architecture.

5.5 Waveform Analysis

The Vivado waveform shows the temporal relationship between the applied input sequence and the detection output. The input bus In[15:0] contains the digital samples applied to the FPGA implementation, while Out3 represents the final detection decision.

During normal operation, the input may contain signal variations and noise without producing a valid detection event. In this condition, Out3 remains at logic low.

When the input undergoes a sufficiently significant transition, the signal passes through the implemented detection chain. If the resulting processed variation satisfies the detection condition and the validation logic accepts the event, Out3 becomes logic high for the corresponding detection interval.

Therefore, the detection output is not expected to reproduce the waveform of the input signal. Instead, it represents an event-based decision. A correct

implementation is consequently characterized by distinct detection pulses associated with the relevant input events.

The waveform also shows that the detection output returns to logic low after the detected event. This behavior is consistent with the intended operation of an event detector and indicates that the output does not remain permanently asserted after a transient detection.

5.6 Verification Under Normal Operating Conditions

Under normal operating conditions, the system should process the input signal without generating an unwanted detection event.

The MATLAB/Simulink reference model provides the expected behavior for this condition. The Detect signal remains low when the signal variations do not satisfy the selected detection criterion.

The corresponding Vivado simulation shows the same functional behavior at Out3. The output remains at logic low during the normal portions of the tested sequence.

This observation provides evidence that the FPGA implementation does not continuously assert the detection output in the presence of the tested normal signal variations.

5.7 Verification of Fault-Related Events

Fault-related signal variations are expected to produce a detection event when the processed signal satisfies the selected detection criterion.

In the MATLAB/Simulink reference simulation, these events appear as distinct pulses on the Detect output. The corresponding Vivado simulation produces pulses on Out3 when the input sequence reaches the conditions associated with the tested events.

The agreement between these two behaviors provides evidence that the generated HDL implementation preserves the intended functional detection mechanism of the reference model.

The functional correspondence is therefore expressed as:

$$\text{Detect}_{\text{MATLAB}} \longleftrightarrow \text{Out3}_{\text{Vivado}}. \quad (5.4)$$

This correspondence is evaluated at the event level rather than by requiring identical numerical representations at every internal processing stage.

5.8 Verification of Repeated Detection Events

The design was also evaluated using sequences containing more than one event. The purpose of this test is to verify that the detector can return to its inactive state after an event and respond again when a subsequent event occurs.

The MATLAB/Simulink reference model produces separate detection pulses for the tested events. The Vivado waveform similarly shows separate high-level

transitions on Out3, with the output returning to logic low between the detected events.

This behavior is important because it demonstrates that the detection mechanism is not permanently latched after the first event under the tested conditions.

The repeated-event behavior therefore provides an additional indication of functional consistency between the reference model and the FPGA implementation.

5.9 Reset and Initialization Verification

The reset signal was also considered during the RTL simulation. The purpose of the reset verification is to ensure that the design starts from a defined state before normal processing begins.

Following reset initialization, the detection output is expected to remain inactive until valid processing conditions are established. The observed behavior is consistent with this requirement for the tested reset sequence.

This verification does not constitute a complete analysis of every possible reset timing relationship. A more exhaustive verification would require additional reset assertion and deassertion scenarios relative to the processing clock.

5.10 Verification Coverage and Limitations

The performed verification provides functional evidence that the FPGA implementation reproduces the intended detection behavior of the MATLAB/Simulink reference model under the tested conditions. In particular, the correspondence between Detect and Out3 was examined for normal operation, fault-related events, repeated events, and reset behavior.

However, the verification has several limitations.

First, the analysis is based on a finite set of test conditions and does not cover every possible input sequence. Second, the top-level waveform does not expose every internal signal generated by the HDL implementation. Consequently, individual internal operations such as the exact FIR output, intermediate difference signal, absolute-value result, gain output, and comparator output are not independently verified through the external waveform.

Third, the visual waveform comparison does not by itself establish exact sample-by-sample equivalence or prove the exact internal latency of every processing stage. Such verification would require a self-checking HDL testbench with explicitly defined expected outputs.

A more extensive future verification strategy could include:

- normal operating sequences;
- noise-dominated sequences;
- single-sample disturbances;
- two-sample events;
- sustained fault events;

- repeated fault events;
- boundary conditions around the detection threshold;
- reset assertion and deassertion at different clock phases;
- automatic comparison between expected and simulated Out3 values.

Such a self-checking verification environment would provide stronger evidence of sample-level equivalence between the MATLAB/Simulink reference behavior and the generated RTL implementation.

5.11 Chapter Summary

This chapter presented the functional verification of the FPGA-based cable harness detection system. The MATLAB/Simulink model was used as the behavioral reference, while the generated VHDL design was evaluated through RTL simulation in Vivado.

The main verification criterion was the correspondence between the MATLAB/Simulink Detect output and the FPGA Out3 output. The waveform analysis showed consistent detection behavior for the tested normal, fault-related, and repeated-event conditions. Reset behavior was also considered during RTL simulation.

The results provide evidence that the FPGA implementation preserves the intended event-detection behavior of the reference model under the tested conditions. The limitations identified in this chapter also define the scope for a more exhaustive self-checking verification environment.

The timing closure, resource utilization, power estimation, and implementation results are presented separately in Chapter 6. ““

Chapter 6

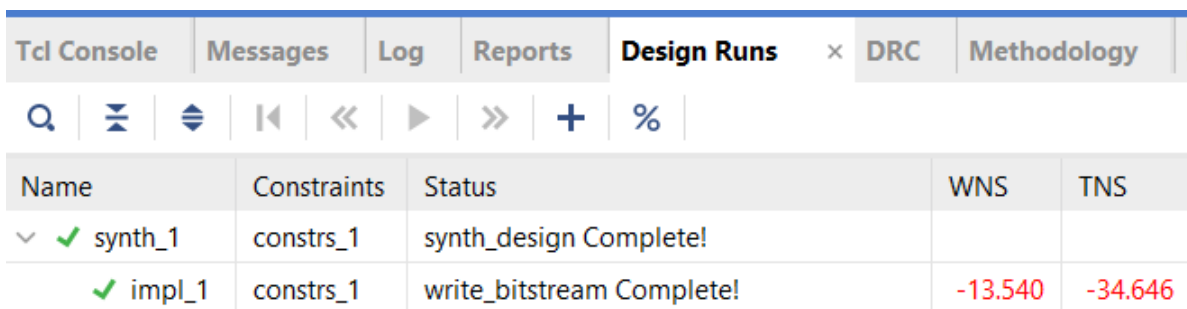
FPGA Implementation Results and Timing Closure

6.1 Initial Vivado Implementation

Following the generation of synthesizable VHDL from the MATLAB/Simulink model, the design was integrated into AMD/Xilinx Vivado and targeted to the Artix-7 XC7A100T FPGA with part number xc7a100tcsg324-1.

The initial implementation was performed using the original system clock configuration. The implementation flow included synthesis, optimization, placement, routing, and timing analysis. The objective of this first implementation was to evaluate whether the generated hardware could satisfy the original timing constraint and to identify potential implementation bottlenecks.

The initial implementation results revealed a significant setup-timing violation. The main timing results are presented in the following section.



The screenshot shows the Vivado IDE interface with the 'Design Runs' tab selected. Below the tabs is a toolbar with icons for search, zoom, and navigation. The main table displays the results of the synthesis and implementation runs.

Name	Constraints	Status	WNS	TNS
✓ synth_1	constrs_1	synth_design Complete!		
✓ impl_1	constrs_1	write_bitstream Complete!	-13.540	-34.646

Figure 6.1: Initial Vivado implementation result before timing optimization.

6.2 Initial Timing Failure

The first implementation did not satisfy the required setup timing constraint. The main timing indicators reported by Vivado were:

- Worst Negative Slack (WNS): -13.540 ns.
- Total Negative Slack (TNS): -34.646 ns.

- Setup timing failing endpoints: 19 out of 1019.
- Hold timing: passed.
- Pulse-width timing: passed.

The negative WNS indicates that the most critical setup path exceeded the available timing budget by 13.540ns. The negative TNS confirms that setup timing violations were present across multiple failing endpoints.

Importantly, the hold and pulse-width checks were successful. Therefore, the initial implementation problem was specifically associated with setup timing rather than with hold-time or clock-pulse-width constraints.

6.3 Diagnosis of the Timing Problem

The initial timing results indicated that the generated processing logic could not reliably operate under the original clock constraint. The negative setup slack showed that the combinational and sequential paths required more time than was available within one clock period.

The problem was therefore addressed at the clock-architecture level rather than by modifying the functional signal-processing algorithm. This approach preserved the validated detection architecture while providing a more realistic timing budget for the generated FPGA implementation.

The analysis showed that increasing the available clock period was an appropriate implementation-level solution for the current design. The functional processing chain remained unchanged.

6.4 Addition of the Clock Management Block

To address the setup-timing violation, a Vivado clock-management block named `clk_wiz_0` was introduced into the FPGA design.

The original system clock was maintained at 100MHz, while a dedicated processing clock was generated for the hardware signal-processing logic. The resulting processing frequency was selected as 40MHz.

The new clock architecture therefore consists of:

- System/input clock: 100 MHz.
- Processing clock: 40 MHz.
- Processing-clock period: 25ns.

This modification increased the available timing budget for the generated processing logic while preserving the intended functionality of the detection system.

6.5 Transition to 40 MHz Processing

The processing logic was subsequently operated using the 40 MHz clock generated by `clk_wiz_0`. The corresponding clock period is:

$$T_{\text{clk}} = \frac{1}{40 \text{ MHz}} = 25 \text{ ns.} \quad (6.1)$$

This provides a substantially larger timing budget than the original 100 MHz clock period of 10 ns.

The signal-processing architecture itself was not fundamentally modified. The FIR filtering, difference computation, absolute-value operation, gain stage, threshold comparison, and two-sample validation logic remained part of the generated hardware implementation.

6.6 Re-Synthesis and Re-Implementation

After modifying the clock architecture, the design was re-synthesized and re-implemented in Vivado. The complete implementation flow was executed again, including synthesis, optimization, placement, routing, and post-implementation timing analysis.

The objective was to verify that the new clock configuration eliminated the setup violations while maintaining the functional architecture of the FPGA design.

The resulting implementation successfully achieved timing closure, as described in the following section.

6.7 Final Timing Closure

The final implementation successfully satisfied the timing constraints. The main post-implementation timing results were:

- Worst Negative Slack (WNS): +0.093 ns.
- Worst Hold Slack (WHS): +0.043 ns.
- Total Negative Slack (TNS): 0 ns.
- Total hold slack violation: 0 ns.
- Failing timing endpoints: 0.

The positive WNS confirms that the worst setup path satisfies the timing requirement. Similarly, the positive WHS confirms that the hold-time requirement is satisfied.

The resulting timing closure demonstrates that the implemented design can satisfy the selected 40 MHz processing-clock constraint.

Based on the reported setup slack, the corresponding critical-path delay can be expressed as:

$$T_{\text{critical}} = 25 \text{ ns} - 0.093 \text{ ns} = 24.907 \text{ ns.} \quad (6.2)$$

An equivalent frequency based on this timing value is approximately:

$$F_{\text{eq}} \approx \frac{1}{24.907 \text{ ns}} \approx 40.15 \text{ MHz.} \quad (6.3)$$

This value is an equivalent frequency derived from the reported timing slack and should not be interpreted as a separately measured maximum operating frequency.

6.8 Resource Utilization

The final implemented design uses a small fraction of the available FPGA resources. The main post-implementation utilization results are summarized below.

Table 6.1: Final FPGA resource utilization.

Resource	Used	Available	Utilization
LUT	744	63,400	1.17%
Flip-Flop	1,039	126,800	0.82%
DSP48	2	240	0.83%
I/O	20	210	9.52%
MMCM	1	6	16.67%

The LUT, flip-flop, and DSP utilization remain very low relative to the available device resources. The MMCM utilization reflects the use of one clock-manage resource for the generation of the 40 MHz processing clock.

The I/O utilization is also limited and remains well within the available device capacity.

Resource	Utilization	Available	Utilization %
LUT	744	63400	1.17
FF	1039	126800	0.82
DSP	2	240	0.83
IO	20	210	9.52
MMCM	1	6	16.67

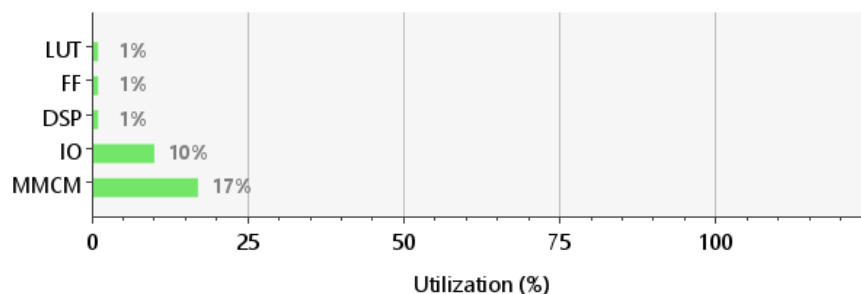


Figure 6.2: Vivado post-implementation resource utilization report.

6.9 Power Analysis

The post-implementation power estimation was also performed in Vivado. The reported total power was approximately 0.208 W.

The main power components were:

- Total power: 0.208 W.
- Dynamic power: 0.110 W.
- Static power: 0.097 W.
- Estimated junction temperature: 25.9°C.

These values correspond to the Vivado power-analysis conditions and should therefore be interpreted as implementation estimates rather than measurements performed using external laboratory equipment.

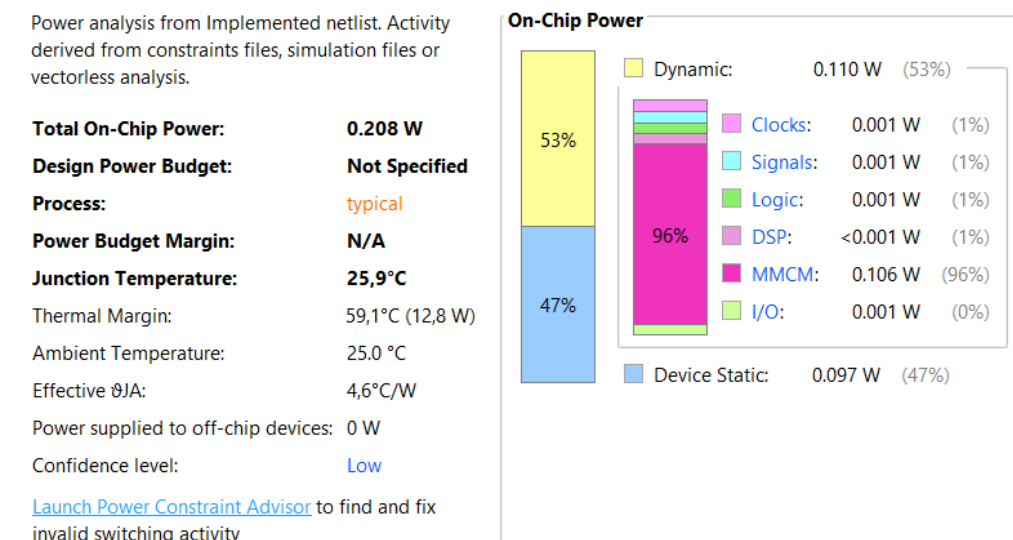


Figure 6.3: Vivado power analysis for the final FPGA implementation.

6.10 Design Rule Check

A post-implementation Design Rule Check (DRC) was performed to identify implementation violations.

The final DRC report contained:

- Errors: 0.
- Critical warnings: 0.
- Warnings: 6.

The reported warnings correspond to DSP pipelining recommendations. They did not result in timing failures or functional implementation errors in the final design.

The absence of DRC errors and critical warnings confirms that the final implemented design passed the relevant Vivado design-rule checks.

6.11 TIMESTAMP Implementation Decision

During the FPGA implementation process, the original design concept included timestamp information associated with detected events. However, an implementation issue was encountered when the timestamp-related logic was included in the FPGA bitstream generation flow.

Rather than introducing additional implementation uncertainty into the final hardware design, the timestamp functionality was removed from the FPGA implementation. After this modification, the FPGA bitstream was successfully generated.

Consequently, the final FPGA interface exposes the detection result through the externally observable signal Out3, while timestamp information remains available only in the MATLAB/Simulink reference model.

This distinction is important for interpreting the verification results. The timestamp functionality was not claimed as part of the final FPGA hardware implementation, and it was therefore not used as an externally observable FPGA verification signal.

6.12 Before and After Clock Modification

The timing improvement obtained by changing the processing clock from the original configuration to 40 MHz is summarized in Table 6.2.

Table 6.2: Timing results before and after the clock modification.

Timing metric	Initial implementation	Final implementation
Processing clock	Original configuration	40 MHz
Clock period	10 ns	25 ns
WNS	-13.540 ns	+0.093 ns
TNS	-34.646 ns	0 ns
WHS	Positive	+0.043 ns
Setup failing endpoints	19 / 1019	0
Hold violations	0	0
Pulse-width violations	0	0

The comparison demonstrates a substantial improvement in setup timing. The WNS changed from -13.540 ns to +0.093 ns, while the number of failing setup endpoints decreased from 19 to zero.

The modification therefore converted the implementation from a setup-timing failure into a timing-closed design.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,093 ns	Worst Hold Slack (WHS): 0,043 ns	Worst Pulse Width Slack (WPWS): 3,000 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2054	Total Number of Endpoints: 2054	Total Number of Endpoints: 1045

All user specified timing constraints are met.

Figure 6.4: Comparison of Vivado timing results before and after the processing-clock modification.

6.13 Discussion of Implementation Results

The final FPGA implementation demonstrates that the generated signal-processing architecture can be successfully implemented on the target Artix-7 device after an appropriate clock-architecture adjustment.

The initial implementation exposed a significant setup-timing limitation. Instead of modifying the functional detection algorithm, the implementation was improved by introducing `clk_wiz_0` and operating the processing logic at 40 MHz. This increased the available clock period from 10 ns to 25 ns and resulted in positive setup and hold slack.

The final implementation also demonstrates low utilization of the main programmable resources. The design consumes only 1.17% of the available LUTs, 0.82% of the flip-flops, and 0.83% of the DSP48 resources. This leaves substantial device capacity for potential future extensions.

The power analysis indicates a total estimated power consumption of 0.208 W under the Vivado analysis conditions. The DRC completed without errors or critical warnings.

The timestamp functionality was deliberately excluded from the final FPGA implementation because its inclusion caused an implementation issue during bitstream generation. Removing this functionality allowed successful bitstream generation while preserving the main externally observable detection function through `Out3`.

Overall, the implementation results confirm that the final hardware architecture is timing-closed, resource-efficient, and suitable for the intended real-time detection function at a 40 MHz processing rate.

6.14 Chapter Summary

This chapter presented the FPGA implementation results and the timing-closure process.

The initial implementation failed the setup-timing requirement, with a WNS of -13.540 ns and TNS of -34.646 ns. Hold and pulse-width checks were nevertheless

successful. The timing issue was addressed by introducing the Vivado Clocking Wizard and generating a 40 MHz processing clock with a 25 ns period.

After re-synthesis and re-implementation, the final design achieved a WNS of +0.093 ns and a WHS of +0.043 ns, with zero failing timing endpoints. The final design also exhibited low LUT, flip-flop, and DSP utilization, an estimated total power of 0.208 W, and zero DRC errors.

The timestamp functionality was not included in the final FPGA hardware because of an implementation issue encountered during bitstream generation. The successful final bitstream therefore exposes the detection function through Out3, while timestamp information remains part of the MATLAB/Simulink reference model.

These results establish the implementation feasibility of the proposed real-time FPGA detection architecture and provide the hardware-level foundation for the functional verification presented in Chapter 5.

Chapter 7

Discussion

7.1 Overall Discussion

The developed system demonstrates the feasibility of implementing a real-time cable harness fault-detection function on an FPGA using a MATLAB/Simulink-based design flow.

The proposed processing chain combines FIR filtering, difference computation, absolute-value calculation, gain, threshold comparison, and two-sample validation. The functional verification presented in Chapter 5 confirmed the expected detection behavior at both the MATLAB/Simulink reference-model level and the Vivado RTL simulation level.

The FPGA implementation results presented in Chapter 6 showed that the initial timing constraint could not be satisfied with the original clock configuration. The introduction of `clk_wiz_0` and the use of a 40 MHz processing clock resolved the setup-timing problem. The final implementation achieved positive setup and hold slack with zero failing timing endpoints.

The implementation also required a practical design decision concerning the timestamp functionality. Because the timestamp-related logic caused an implementation issue during bitstream generation, it was excluded from the final FPGA hardware. The main detection function was retained through the externally observable `Out3` signal.

The implementation also required a practical design decision concerning the timestamp functionality. Because the timestamp-related logic caused an implementation issue during bitstream generation, it was excluded from the final FPGA hardware. The main detection function was retained through the externally observable `Out3` signal.

7.2 Main Strengths and Limitations

The main strength of the implementation is the complete design flow from MATLAB/Simulink modeling to synthesizable VHDL, Vivado implementation, timing analysis, and RTL verification on the target FPGA architecture.

The final design also uses a relatively small portion of the available FPGA resources. This leaves room for future extensions of the processing and monitoring architecture.

However, the present implementation remains a functional prototype. The complete physical cable-harness acquisition chain, including the analog front-end and ADC, was not physically implemented in this project. In addition, the FPGA implementation exposes the detection result but does not currently provide the timestamp functionality that was available in the reference model.

Therefore, the obtained results demonstrate the feasibility of the digital detection architecture rather than a complete industrial measurement system.

7.3 Discussion Summary

Overall, the results show that the proposed FPGA architecture can perform the intended real-time detection function while satisfying the final 40 MHz implementation constraint. The work also identifies the main remaining steps required to transform the current prototype into a more complete hardware demonstrator.

Chapter 8

Future Work

8.1 Hardware and Acquisition Improvements

A first extension would be the implementation of the complete physical acquisition chain, including the signal-conditioning stage, multiplexing or switching circuitry, ADC interface, and the cable-harness connections.

This would allow the FPGA processing architecture to be evaluated using physical measurements rather than only simulated input data.

8.2 FPGA Architecture Improvements

The FPGA design could also be extended with additional monitoring and event-management functionality. In particular, the timestamp mechanism could be redesigned and integrated more carefully into the hardware architecture so that detected events can be associated with their occurrence time without affecting bitstream generation or timing closure.

Additional hardware interfaces could also be introduced to expose more internal information for debugging and system monitoring.

8.3 Detection Algorithm Improvements

The current detection parameters were selected from the tested simulation conditions. A more extensive study could evaluate the detector using a larger set of representative fault and noise conditions.

Future work could therefore include systematic parameter optimization and quantitative evaluation using labeled datasets and metrics such as detection rate, false-positive rate, precision, recall, and detection latency.

8.4 System-Level Extensions

The FPGA detector could ultimately be integrated into a larger industrial monitoring architecture. Communication, data logging, diagnostic interfaces, and higher-level condition-monitoring functions could be added around the FPGA processing core.

Such extensions would provide a path from the current FPGA prototype toward a more complete cyber-physical monitoring system.

8.5 Future Work Summary

The main future developments therefore concern physical signal acquisition, improved FPGA event management, systematic evaluation of the detection algorithm, and integration with a complete industrial monitoring system.

Chapter 9

Conclusion

This project presented the design and FPGA implementation of a real-time cable harness fault-detection system based on MATLAB/Simulink, HDL Coder, VHDL, and AMD/Xilinx Vivado.

The developed digital architecture implements a signal-processing chain consisting of FIR filtering, difference calculation, absolute-value computation, gain, threshold comparison, and two-sample validation. The functional behavior was verified using both the MATLAB/Simulink reference model and Vivado RTL simulation.

The initial FPGA implementation revealed a significant setup-timing problem. This issue was addressed by introducing `clk_wiz_0` and using a 40 MHz processing clock with a 25 ns period. The final implementation achieved a WNS of +0.093 ns and a WHS of +0.043 ns, with zero failing timing endpoints.

The final design also demonstrated low utilization of the available FPGA resources and an estimated total power consumption of 0.208 W under the Vivado analysis conditions. The final DRC contained no errors or critical warnings.

A practical limitation concerned the timestamp functionality. Because its inclusion caused an implementation issue during bitstream generation, the timestamp was removed from the final FPGA implementation. The main detection function was retained through the externally observable `Out3` signal.

The work therefore establishes a functional and timing-closed FPGA implementation of the proposed digital detection architecture. Further development can focus on physical signal acquisition, improved event management, quantitative detection evaluation, and integration into a complete industrial monitoring system.

Bibliography

- [1] AMD/Xilinx, *Vivado Design Suite Documentation*.
- [2] MathWorks, *HDL Coder Documentation*.
- [3] MathWorks, *Simulink Documentation*.